TEO_finale

January 26, 2026

N.B. this notebook has been tested in Google Colab

1 Music Genre Classification

In this project, you will build a machine learning algorithm to classify music genres using audio features. Starting with the provided dataset, your task is to develop a model that effectively solves this multiclass classification problem. Use the baseline notebook as a starting point and improve upon it.

Overall Goal: to design a complete pipeline that improves accuracy from the current 37% to at least 70%, ideally approaching 80%

1.1 Introduction

In this project, you will work with a dataset of music samples from various genres. The dataset has been purposely left a bit messy, with some entries missing labels and others containing empty audio files. To start with, your task is to clean and explore the dataset, turning it into a well-organized resource for analysis.

This notebook includes a basic, "weak" baseline to get you started. It serves as a simple starting point, but it is neither thorough nor accurate. You are expected to build upon it, applying your own strategies to improve the data science pipeline (including data cleaning, curation, feature engineering, etc) before moving into model building, parameters tuning, and model evaluation.

The formal details of the assignment are provided at the end of the notebook. To start with, focus on understanding the dataset and planning your strategies to tackle its challenges.

We expect you to submit a modified version of this notebook with your improvements. Please download a copy of this assignment in your private Python programming environment, before making any changes.

1.2 Baseline

Let's install all required dependencies:

- **datasets**: Access to large-scale datasets.
- **librosa**: Tools for audio analysis.
- **pandas** & **numpy**: Tabular data manipulation and numerical operations.
- **scikit-learn**: Machine learning algorithms and tools.
- **tqdm**: Progress bar.

You might be familiar with most of these already.

[1]:
```
%%capture
%pip install datasets==3.5.0 librosa pandas numpy scikit-learn tqdm
```

And import the necessary modules keras - https://keras.io/ librosa - https://librosa.github.io/librosa/ matplotlab - https://matplotlib.org/ pandas - https://pandas.pydata.org/ tensorflow (Requires Python 3.6) - https://www.tensorflow.org/

keras – High-level API to build and train neural networks. librosa – Audio signal processing and feature extraction for music. matplotlib – Visualization of data, training curves, and results. pandas – Data handling and organization of features and labels. tensorflow – Backend framework for numerical computation and deep learning. numpy – Numerical computing with arrays and matrices, used for audio data and feature manipulation.

[2]:
```
import csv
import datetime
import hashlib
import math
import random
import statistics
from random import randint, seed
from itertools import combinations

import numpy as np
import pandas as pd
import librosa
import librosa.display
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
from matplotlib.collections import PolyCollection

from datasets import load_dataset, Dataset
from IPython.display import Audio, display
from tqdm.auto import tqdm

from scipy import signal

import sklearn
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import DBSCAN
from sklearn import preprocessing
from sklearn.preprocessing import (
    OneHotEncoder,
    LabelEncoder,
    StandardScaler
)
```

```
from sklearn.model_selection import (
    train_test_split,
    GridSearchCV,
    RepeatedStratifiedKFold,
    cross_val_score
)

from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score
)

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

import keras
from keras import models, layers

import seaborn as sns
from sklearn.model_selection import StratifiedKFold
```

2 DATA EXPLORATION AND CURATION

2.0.1 Dataset Description

The dataset consists of music samples from various genres, including:

- **Genres**: `Blues`, `Classical`, `Country`, `Disco`, `HipHop`, `Jazz`, `Metal`, `Pop`, `Reggae`, and `Rock`.

The dataset is a bit messy and includes some **unlabeled data** and **empty audio files**. We have provided basic preprocessing, but more in-depth data cleaning, feature extraction, and preparation will be a part of your assignment.

Let's download the audio dataset using the Hugging Face datasets library.

[3]:
```
dataset = load_dataset("unibz-ds-course/audio_assignment", split="train")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
```

```
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab
and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(
```

```
README.md:   0%|          | 0.00/643 [00:00<?, ?B/s]

data/train-00000-of-00004.parquet:   0%|          | 0.00/312M [00:00<?, ?B/s]

data/train-00001-of-00004.parquet:   0%|          | 0.00/336M [00:00<?, ?B/s]

data/train-00002-of-00004.parquet:   0%|          | 0.00/309M [00:00<?, ?B/s]

data/train-00003-of-00004.parquet:   0%|          | 0.00/318M [00:00<?, ?B/s]

Generating train split:   0%|          | 0/1230 [00:00<?, ? examples/s]
```

[4]:
```
dataset
```

[4]:
```
Dataset({
    features: ['file', 'audio', 'genre'],
    num_rows: 1230
})
```

[5]:
```
print(f"Num of samples in the dataset: {len(dataset)}")
```

```
Num of samples in the dataset: 1230
```

Let's take a glance at a sample from the dataset

[6]:
```
entry = dataset[10]

audio_array = entry['audio']['array']
sampling_rate = entry['audio']['sampling_rate']

print(f"Element: {entry}")
print(f"File Path: {entry['file']}")
print(f"Number of Samples: {len(audio_array)}")
print(f"Sampling Rate: {sampling_rate} Hz")

audio_length_seconds = len(audio_array) / sampling_rate
print(f"Audio Length: {audio_length_seconds:.2f} seconds")

genre_id = entry['genre']
genre_label = dataset.features['genre'].int2str(genre_id)
print(f"Genre (ID): {genre_id}")
print(f"Genre (Label): {genre_label}")

display(Audio(audio_array, rate=sampling_rate))
```

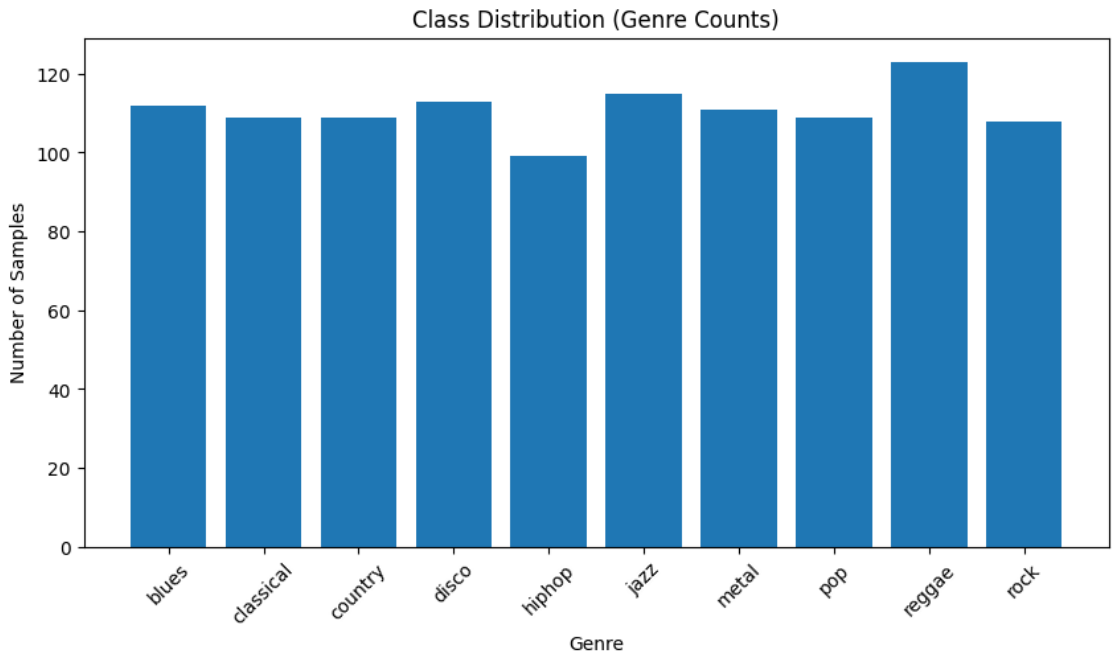
```
Element: {'file': 'audio_data/country/country.00247.wav', 'audio': {'path':
'country.00247.wav', 'array': array([-0.01055908, -0.01712036, -0.02111816, …,
0.03747559,
        0.01239014, -0.03366089]), 'sampling_rate': 22050}, 'genre': 2}
File Path: audio_data/country/country.00247.wav
Number of Samples: 661794
Sampling Rate: 22050 Hz
Audio Length: 30.01 seconds
Genre (ID): 2
Genre (Label): country
```

```
<IPython.lib.display.Audio object>
```

Draw a plot with distribution of the classes (5 points)

Create a visualization (e.g., bar chart or histogram) that shows how many samples belong to each class. This helps identify whether the dataset is balanced or if some classes are underrepresented.

[7]:
```python
df = dataset.to_pandas()

# Remove missing labels
df = df.dropna(subset=['genre'])

# Convert to int
df['genre'] = df['genre'].astype(int)

# Count
counts = df['genre'].value_counts().sort_index()

# Map to names
genre_names = [dataset.features['genre'].int2str(i) for i in counts.index]

# Plot
plt.figure(figsize=(10, 5))
plt.bar(genre_names, counts)
plt.xticks(rotation=45)
plt.title("Class Distribution (Genre Counts)")
plt.xlabel("Genre")
plt.ylabel("Number of Samples")
plt.show()
```

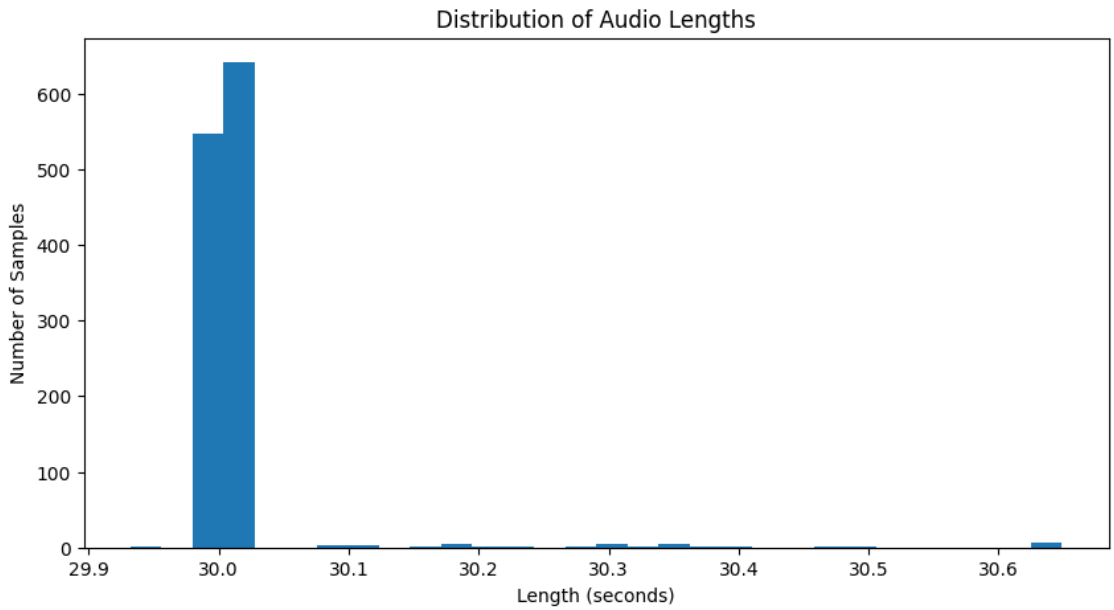


This graph helpd visualize whether there are class imbalances. At this point there are no severe imbalances.

Draw distribution of lengths of audios (5 points)

Plot the distribution of audio lengths in the dataset to analyze how durations vary across samples.

[8]:
```
# compute length in seconds for each audio sample
lengths = [
    entry['audio']['array'].shape[0] / entry['audio']['sampling_rate']
    for entry in dataset
    if entry['audio'] is not None
]

# plot histogram
plt.figure(figsize=(10, 5))
plt.hist(lengths, bins=30)
plt.title("Distribution of Audio Lengths")
plt.xlabel("Length (seconds)")
plt.ylabel("Number of Samples")
plt.show()
```



Also in this case there is no severe class imbalance

2.0.2 Delete empty samples (5 points)

Implement the function to remove empty samples (audios with silence only)

[9]:
```
def filter_empty_samples(entry):
    audio = entry["audio"]

    if audio is None:
        return False

    arr = audio["array"]
    if arr is None or len(arr) == 0:
        return False

    # Check silence (all zeros)
    if (arr == 0).all():
        return False

    return True


filtered_dataset = dataset.filter(filter_empty_samples)

print("Original:", len(dataset))
print("Filtered:", len(filtered_dataset))
```

```
Filter:   0%|          | 0/1230 [00:00<?, ? examples/s]

Original: 1230
Filtered: 970
```

[10]:
```python
filtered_dataset = dataset.filter(filter_empty_samples)
```

2.0.3 Delete unlabeled samples (5 points)

Implement the function to remove unlabeled samples.

[11]:
```python
def filter_valid_samples(entry):
    # remove unlabeled
    genre = entry["genre"]
    if genre is None or (isinstance(genre, float) and math.isnan(genre)):
        return False

    # remove empty audio
    audio = entry["audio"]
    if audio is None:
        return False
    arr = audio["array"]
    if arr is None or len(arr) == 0:
        return False
    if np.all(arr == 0):
        return False

    return True

filtered_dataset = dataset.filter(filter_valid_samples)
print(len(filtered_dataset))
```

```
Filter:   0%|          | 0/1230 [00:00<?, ? examples/s]

848
```

2.0.4 Delete duplicated audios

Duplicate removal is carried out in two stages, each addressing a different source of redundancy in the dataset. The first stage operates directly on the raw audio data. For each audio file, a compact signature is computed using summary statistics of MFCCs (mean and standard deviation) and the signal duration. These signatures are then clustered with DBSCAN using a very small distance threshold, so that only virtually identical recordings are grouped together. From each group, a single representative sample is retained. This step removes duplicated or re-encoded versions of the same audio track before feature extraction, preventing information leakage between training and test sets.

The second stage is applied after feature extraction and feature merging. Here, duplicates are identified by checking for identical rows in the final feature matrix. Samples with exactly the same feature vectors are grouped together. If all samples in a group share the same genre label,

only the first occurrence is kept; if conflicting genre labels are found, all samples in that group are removed to avoid introducing label noise. This stricter, feature-level deduplication acts as a final consistency check, ensuring that no duplicate observations—either accidental or caused by preprocessing artifacts—remain in the dataset used for modeling. Together, these two steps improve data integrity, reduce overfitting risk, and lead to a more reliable evaluation of model performance.

FIRST STAGE DUPLICATE REMOVAL

[12]:
```
def compute_signature(audio):
    arr = audio["array"]
    sr = audio["sampling_rate"]

    mfcc = librosa.feature.mfcc(y=arr, sr=sr, n_mfcc=20)
    mfcc_mean = mfcc.mean(axis=1)
    mfcc_std = mfcc.std(axis=1)
    duration = len(arr) / sr

    return np.concatenate([mfcc_mean, mfcc_std, [duration]])


rows = []
for i in range(len(filtered_dataset)):
    e = filtered_dataset[i]
    sig = compute_signature(e["audio"])
    rows.append({
        "idx": i,
        "signature": sig,
        "genre": int(e["genre"]),
        "file": e["file"]
    })

df = pd.DataFrame(rows)

X = np.stack(df["signature"].values)

cl = DBSCAN(eps=1e-5, min_samples=1, metric="euclidean")
labels = cl.fit_predict(X)

df["group"] = labels

grouped = df.groupby("group").agg({
    "genre": lambda x: x.mode()[0],
    "idx": "first"
}).reset_index()

keep_indices = grouped["idx"].tolist()

# **this is the deduplicated dataset**
```

```
filtered_dataset = filtered_dataset.select(keep_indices)

print("after dedup:", len(filtered_dataset))
```

```
after dedup: 828
```

2.1 Remove noise only audios

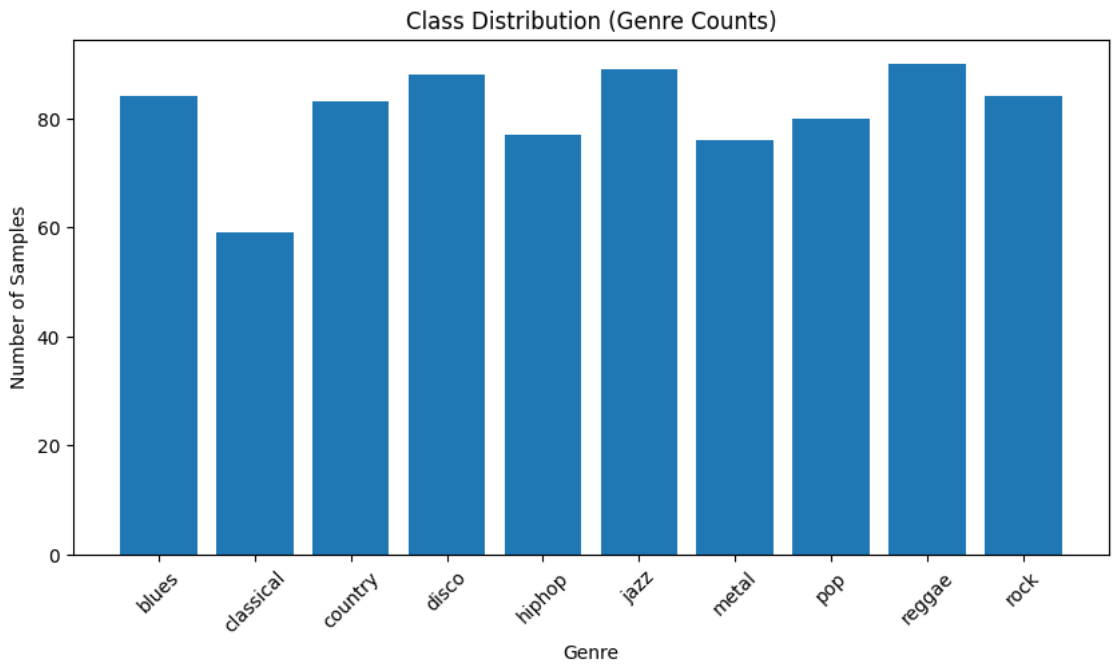
[13]:
```
def is_music_like(audio, rms_thresh=0.02, flatness_thresh=0.2):
    arr = audio["array"]

    rms = librosa.feature.rms(y=arr).mean()
    flatness = librosa.feature.spectral_flatness(y=arr).mean()

    return (rms > rms_thresh) and (flatness < flatness_thresh)

def filter_music_samples(entry):
    audio = entry["audio"]
    if audio is None:
        return False

    arr = audio["array"]
    if arr is None or len(arr) == 0:
        return False

    return is_music_like(audio)

# **apply filter on the already deduplicated dataset**
filtered_dataset = filtered_dataset.filter(filter_music_samples)

print("final:", len(filtered_dataset))
```

```
Filter:   0%|          | 0/828 [00:00<?, ? examples/s]

final: 810
```

Now I want to check again whether there are class imbalances

[14]:
```
df = filtered_dataset.to_pandas()

# Convert to int
df['genre'] = df['genre'].astype(int)

# Count
counts = df['genre'].value_counts().sort_index()

# Map to names
genre_names = [dataset.features['genre'].int2str(i) for i in counts.index]
```

```
# Plot
plt.figure(figsize=(10, 5))
plt.bar(genre_names, counts)
plt.xticks(rotation=45)
plt.title("Class Distribution (Genre Counts)")
plt.xlabel("Genre")
plt.ylabel("Number of Samples")
plt.show()
```



[15]:
```
import pandas as pd

# Convert dataset to pandas once
df = filtered_dataset.to_pandas()

# Count by genre ID
counts = df["genre"].value_counts().sort_index()

# Get genre names from dataset features
genre_names = filtered_dataset.features["genre"].names

# Convert to name-based table
table = pd.DataFrame({
    "genre_id": counts.index,
    "genre_name": [genre_names[i] for i in counts.index],
```

```
    "count": counts.values
}).sort_values("count", ascending=False)

print(table)
```

```
   genre_id genre_name  count
8         8     reggae     90
5         5       jazz     89
3         3      disco     88
0         0      blues     84
9         9       rock     84
2         2    country     83
7         7        pop     80
4         4     hiphop     77
6         6      metal     76
1         1  classical     59
```

Now the classes are unbalanced, mainly due to the classical genre. When the model training will be done after all feature extraction, oversampling method will be done

3 FEATURE EXTRACTION

3.0.1 Mel Frequency Cepstral Coefficients

Mel Frequency Cepstral Coefficients (MFCCs) are commonly used in audio analysis to capture key features of sound. They help represent the important characteristics of an audio signal, making them ideal for tasks like music genre classification and speech recognition.

We're not going to dive deep into the complex details of audio processing, but it's useful to know that MFCCs help simplify raw audio data while retaining important information.

Basic Steps in MFCC Extraction:

1. **Frequency Domain Conversion**: The audio signal is split into short frames, and we apply the Fourier Transform to convert them from the time domain to the frequency domain.
2. **Mel Scale Mapping**: The frequency spectrum is converted to the Mel scale, which better represents how humans perceive sound, emphasizing lower frequencies.
3. **Logarithm and DCT**: After mapping to the Mel scale, we apply a logarithm and the Discrete Cosine Transform (DCT) to get the MFCCs. These summarize the "cepstral" information of the audio signal.

The parameter `n_mfcc` controls **how many MFCC coefficients** are extracted for each frame. For example, setting `n_mfcc=8` means we extract 8 coefficients, where lower coefficients capture broad audio features, and higher coefficients capture the more finer details.

In this notebook, we'll use the **mean** and **variance** of the MFCCs over time to create a robust feature set for our classification model. Adjusting the `n_mfcc` parameter allows us to control the number of features extracted for each audio sample.

Additional Features Consider exploring additional audio features to enhance your model's performance. There are various acoustic properties you could extract from the audio signals, such as zero crossings, harmonic-percussive separation, tempo, spectral centroids, spectral rolloff, chromagram, RMS energy, spectral bandwidth, etc. When working with these features, it's often useful to compute summary statistics like the mean and variance across the audio sample. These summary statistics can capture the overall characteristics and variability of the feature, reducing the dimensionality of your data while retaining important information. Experimenting with these features and their statistical summaries could potentially improve your model's accuracy and robustness in distinguishing between different audio characteristics.

Feature Analysis Don´t forget to optimize the use of features, identifying and handling irrelevant and reduntant features. Then use feature ranking to identify which features are more influential, and evaluate quantitatively how many top-features to retain.

[16]:
```
def extract_mfcc_features(dataset, n_mfcc):
    mfcc_features = []

    # here we might have used Dataset.map method, unfortunately, it consumes␣
↪extra memory and runs out of RAM in colab
    for entry in tqdm(dataset, desc="Extracting MFCC Features"):
        audio_array = entry['audio']['array']
        sampling_rate = entry['audio']['sampling_rate']

        mfcc = librosa.feature.mfcc(y=audio_array, sr=sampling_rate,␣
↪n_mfcc=n_mfcc)

        mfcc_mean = np.mean(mfcc, axis=1)
        mfcc_var = np.var(mfcc, axis=1)

        feature_dict = {}

        for i in range(n_mfcc):
            feature_dict[f'mfcc_mean{i+1}'] = mfcc_mean[i]
            feature_dict[f'mfcc_var{i+1}'] = mfcc_var[i]

        feature_dict['genre'] = entry['genre']

        mfcc_features.append(feature_dict)

    return pd.DataFrame(mfcc_features)
```

[17]:
```
# Store all MFCC dataframes in a dict
dfs_mfcc = {}
for n in [8, 13, 20]:
    df_mfcc = extract_mfcc_features(filtered_dataset, n_mfcc=n)
    dfs_mfcc[n] = df_mfcc
    print(n, df_mfcc.shape)
```

```
Extracting MFCC Features:   0%|          | 0/810 [00:00<?, ?it/s]

8 (810, 17)

Extracting MFCC Features:   0%|          | 0/810 [00:00<?, ?it/s]

13 (810, 27)

Extracting MFCC Features:   0%|          | 0/810 [00:00<?, ?it/s]

20 (810, 41)
```

Let's take a look at the output of the function. We will pass there just 2 samples from the dataset.

[18]:
```
extract_mfcc_features(dataset.select(range(2)), n_mfcc=5)
```

```
Extracting MFCC Features:   0%|          | 0/2 [00:00<?, ?it/s]
```

[18]:
```
   mfcc_mean1    mfcc_var1  mfcc_mean2   mfcc_var2  mfcc_mean3   mfcc_var3  \
0  -69.836278  2341.101347  124.207701  580.461335  -32.482371  329.799566
1 -191.643297  3301.314391   41.318241  403.273174    5.013655  447.922087

   mfcc_mean4   mfcc_var4  mfcc_mean5   mfcc_var5  genre
0   47.689801  167.005379    1.400885  125.102031      2
1   65.453821  220.996266   -4.320506  210.413597      4
```

The function generates `n_mfcc * 2` features for each sample. Consider analyzing their correlation with a matrix and experimenting with different `n_mfcc` values to observe how feature relationships change. While MFCC features are effective for audio analysis, you might also improve performance by incorporating additional features such as RMS or Spectral Contrast. Once you've explored these options, proceed to training the model using the extracted features.

I want to check which n_mfcc is best to keep.

[19]:
```
results = []

for n_mfcc, df_mfcc in dfs_mfcc.items():
    df = df_mfcc.copy()
    df["genre"] = df["genre"].astype(int)

    X = df.drop(columns=["genre"]).values
    y = df["genre"].values

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )

    pipeline = Pipeline([
```

```
        ("scaler", StandardScaler()),
        ("lr", LogisticRegression(max_iter=5000, random_state=42))
    ])

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)

    results.append({
        "n_mfcc": n_mfcc,
        "test_acc": accuracy_score(y_test, y_pred),
        "test_f1_macro": f1_score(y_test, y_pred, average="macro")
    })

results_df = pd.DataFrame(results).sort_values("test_acc", ascending=False)
print(results_df)
```

```
   n_mfcc  test_acc  test_f1_macro
2      20  0.709877       0.710107
1      13  0.660494       0.666927
0       8  0.592593       0.584484
```

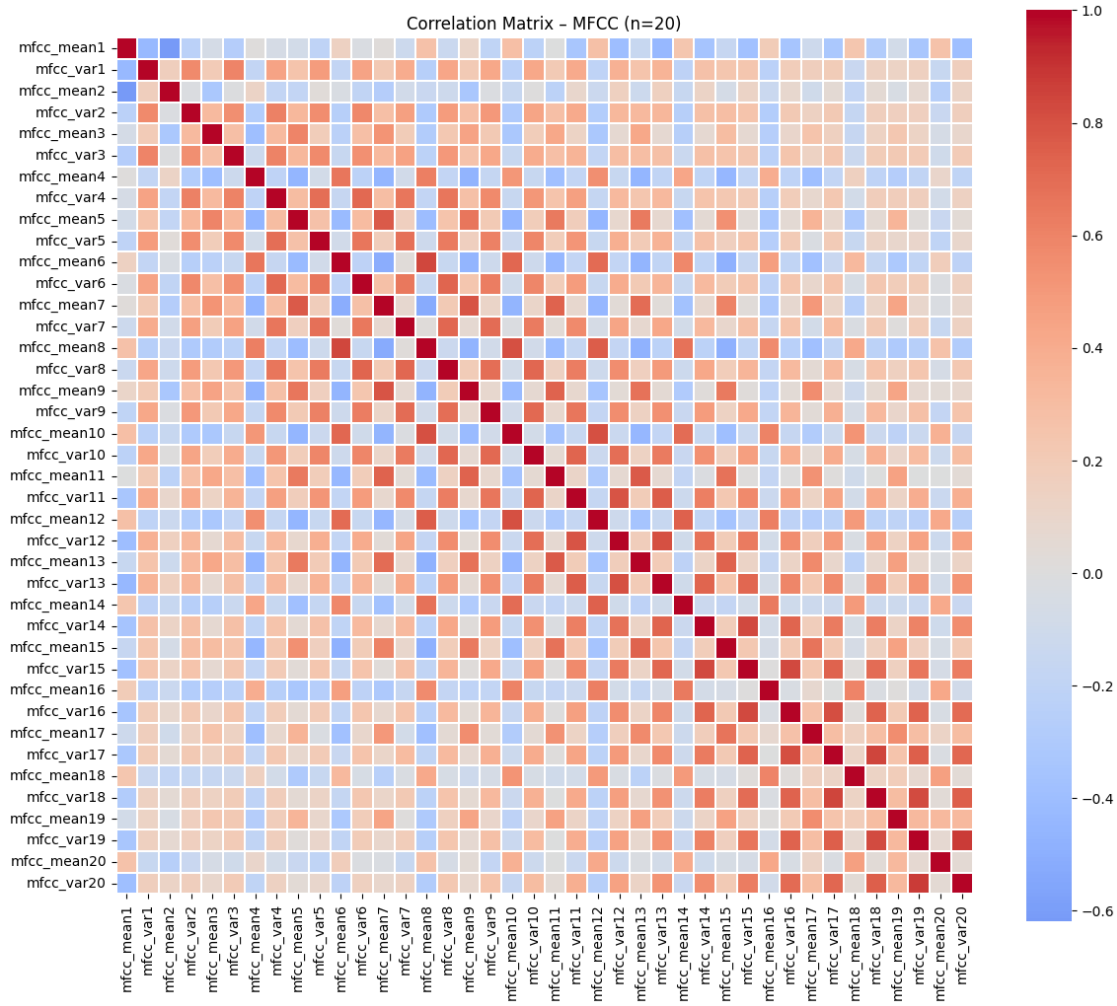
n_mfcc = 20 performs best

4 CORRELATION MATRIX

If mfcc_meanX and mfcc_varX are strongly correlated, then they carry very similar information, so one of them might be unnecessary.

[20]:
```
# select MFCC = 20
df_corr = dfs_mfcc[20].copy()
df_corr["genre"] = df_corr["genre"].astype(int)

# drop label
X_corr = df_corr.drop(columns=["genre"])

# correlation matrix
corr = X_corr.corr()

plt.figure(figsize=(14, 12))
sns.heatmap(
    corr,
    cmap="coolwarm",
    center=0,
    square=True,
    linewidths=0.2
)
plt.title("Correlation Matrix - MFCC (n=20)")
```

```
plt.show()
```



[21]:
```
# --- prepare train matrix (same as before) ---
df_best = dfs_mfcc[20].copy()
df_best["genre"] = df_best["genre"].astype(int)

X = df_best.drop(columns=["genre"])
y = df_best["genre"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

smote = SMOTE(random_state=42)
X_train_os, y_train_os = smote.fit_resample(X_train, y_train)
```

```
# --- correlation matrix ---
df_train_os = pd.DataFrame(X_train_os, columns=X.columns)
corr = df_train_os.corr().abs()

# --- get top correlated pairs ---
pairs = (
    corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
        .stack()
        .reset_index()
        .rename(columns={"level_0": "feature_1", "level_1": "feature_2", 0:␣
 ↪"corr"})
        .sort_values("corr", ascending=False)
)

# show top 20
top20 = pairs.head(20)
print(top20)
```

```
       feature_1    feature_2      corr
778   mfcc_var19   mfcc_var20  0.856427
760   mfcc_var17   mfcc_var18  0.847949
703   mfcc_var14   mfcc_var15  0.841108
348   mfcc_mean6   mfcc_mean8  0.840889
771   mfcc_var18   mfcc_var19  0.837335
745   mfcc_var16   mfcc_var17  0.831722
726   mfcc_var15   mfcc_var16  0.811220
645   mfcc_var12   mfcc_var13  0.808026
458   mfcc_mean8  mfcc_mean10  0.804960
610   mfcc_var11   mfcc_var12  0.796242
552  mfcc_mean10  mfcc_mean12  0.795263
405   mfcc_mean7   mfcc_mean9  0.790291
749   mfcc_var16   mfcc_var19  0.772354
762   mfcc_var17   mfcc_var19  0.772234
462   mfcc_mean8  mfcc_mean12  0.769841
593  mfcc_mean11  mfcc_mean13  0.768245
287   mfcc_mean5   mfcc_mean7  0.765934
612   mfcc_var11   mfcc_var13  0.763066
728   mfcc_var15   mfcc_var17  0.752442
747   mfcc_var16   mfcc_var18  0.751073
```

Many features are highly correlated. The next step is to check whether removing highly correlated features will improve the accuracy. The threshold set is >0.80

[22]:
```
def remove_correlated_features(df, threshold=0.8):
    corr_matrix = df.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).
 ↪astype(bool))
    to_drop = [col for col in upper.columns if any(upper[col] > threshold)]
```

```python
    return df.drop(columns=to_drop), to_drop

# --- remove features ---
X_train_clean, dropped = remove_correlated_features(df_train_os, threshold=0.8)

print("Dropped features:", dropped)
print("Remaining features:", X_train_clean.shape[1])

# apply same drop to test set
X_test_clean = pd.DataFrame(X_test, columns=X.columns).drop(columns=dropped)

def run_logistic(X_tr, y_tr, X_te, y_te):
    pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=5000, random_state=42))
    ])
    pipe.fit(X_tr, y_tr)
    y_pred = pipe.predict(X_te)
    return accuracy_score(y_te, y_pred)

acc_before = run_logistic(df_train_os, y_train_os, X_test, y_test)
acc_after  = run_logistic(X_train_clean, y_train_os, X_test_clean, y_test)

print("\nLogistic Regression accuracy:")
print("Before removal:", acc_before)
print("After removal:", acc_after)
```

```
Dropped features: ['mfcc_mean8', 'mfcc_mean10', 'mfcc_var13', 'mfcc_var15',
'mfcc_var16', 'mfcc_var17', 'mfcc_var18', 'mfcc_var19', 'mfcc_var20']
Remaining features: 31

Logistic Regression accuracy:
Before removal: 0.6975308641975309
After removal: 0.6604938271604939
```

Removing correlated features reduced accuracy. This means that those features can still be informative.

4.1 RMS, SPECTRAL CONTRAST

Implement functions to extract RMS and Spectral Contrast (10 points in total)

Write functions to extract these features, also add their description and discuss why they might be useful in this assignment.

4.2 SPECTRAL CONTRAST

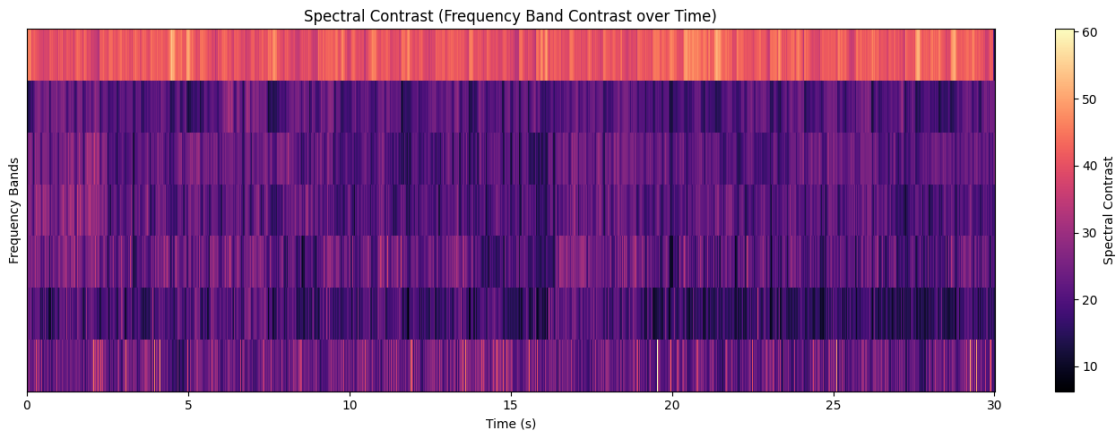
Spectral contrast is an audio feature that measures the difference in energy between the peaks and valleys of the frequency spectrum. Unlike MFCCs, which describe the general shape of the spectral envelope, spectral contrast focuses on the contrast between high-energy and low-energy frequency bands. This makes it particularly effective for capturing the harmonic structure and timbral characteristics of audio signals.

In the context of music genre classification, spectral contrast provides complementary information to MFCCs. MFCCs primarily capture overall spectral shape and broad tonal features, whereas spectral contrast is more sensitive to instrumentation and percussive vs harmonic content. For example, genres such as classical or jazz typically exhibit clearer harmonic structure (higher contrast), while electronic and hip-hop music often show less contrast due to strong bass and noisy components.

Therefore, incorporating spectral contrast into the feature set can improve classification performance by adding information about harmonic richness, brightness, and percussiveness, which are often key differentiators between musical genres. In summary, spectral contrast is a valuable addition to MFCC features because it enhances the model's ability to distinguish genres based on timbre and sound texture.

4.2.1 Demonstrative plot

[23]:
```
entry = filtered_dataset[0]
x = entry["audio"]["array"]
sr = entry["audio"]["sampling_rate"]

# Compute spectral contrast
spectral_contrast = librosa.feature.spectral_contrast(y=x, sr=sr)

plt.figure(figsize=(14, 5))
librosa.display.specshow(
    spectral_contrast,
    x_axis="time",
    cmap="magma"
)
plt.colorbar(label="Spectral Contrast")
plt.title("Spectral Contrast (Frequency Band Contrast over Time)")
plt.xlabel("Time (s)")
plt.ylabel("Frequency Bands")
plt.tight_layout()
plt.show()
```



How to read the plot: Y-axis → frequency bands (low → high) Color intensity → contrast strength: - bright = strong peak–valley separation - dark = flat/noisy spectrum

Genre intuition: - Classical / Jazz → strong contrast in low–mid bands (harmonics, instruments) - Metal / Rock → reduced contrast (distortion fills valleys) - HipHop / Reggae → strong low-band contrast (bass vs silence)

[24]:
```
def extract_spectral_contrast(dataset, n_bands=6):
    contrast_features = []

    for entry in tqdm(dataset, desc="Extracting Spectral Contrast"):
        audio_array = entry['audio']['array']
        sampling_rate = entry['audio']['sampling_rate']

        contrast = librosa.feature.spectral_contrast(y=audio_array,␣
↪sr=sampling_rate, n_bands=n_bands)

        # mean + var for each band
        contrast_mean = np.mean(contrast, axis=1)
        contrast_var = np.var(contrast, axis=1)

        feature_dict = {}
        for i in range(n_bands + 1):  # n_bands+1 bands returned by librosa
            feature_dict[f'contrast_mean{i+1}'] = contrast_mean[i]
            feature_dict[f'contrast_var{i+1}'] = contrast_var[i]

        feature_dict['genre'] = entry['genre']
        contrast_features.append(feature_dict)

    return pd.DataFrame(contrast_features)
```

Checking if it works properly

[25]:
```
df_spcon = extract_spectral_contrast(filtered_dataset)
```

```
Extracting Spectral Contrast:   0%|          | 0/810 [00:00<?, ?it/s]
```

[26]:
```
# ==========================
# 1) NaN check
print("NaNs:", df_spcon.isna().sum().sum())

# 2) Finite check
print("All finite:", np.isfinite(df_spcon.drop(columns=["genre"])).all().all())

# 3) Shape
print("Shape:", df_spcon.shape)

# 4) Descriptive stats
print("\nDescriptive stats:")
print(df_spcon.drop(columns=["genre"]).describe())

# 5) Feature variances
print("\nFeature variances:")
print(df_spcon.drop(columns=["genre"]).var())

# 6) Class-wise mean for all contrast bands
print("\nClass-wise mean for all contrast bands:")

n_bands = 6
for i in range(1, n_bands + 2):  # because bands = n_bands + 1
    col_mean = f"contrast_mean{i}"
    col_var = f"contrast_var{i}"

    print(f"\n--- Band {i} ---")
    print("Mean:", df_spcon.groupby("genre")[col_mean].mean())
    print("Var:", df_spcon.groupby("genre")[col_var].mean())

# 7) Min / Max values
print("\nMin / Max values:")
print(df_spcon.drop(columns=["genre"]).agg(["min", "max"]))
```

```
NaNs: 0
All finite: True
Shape: (810, 15)

Descriptive stats:
       contrast_mean1  contrast_var1  contrast_mean2  contrast_var2  \
count      810.000000     810.000000      810.000000     810.000000
mean        21.840263      23.832422       16.242985      19.280100
std          4.067996       7.642255        2.196789       5.475387
min         13.499980       7.625196       11.713613       9.969447
```

```
25%         18.938024      18.769531       14.685534      15.710508
50%         21.145116      23.150680       15.872416      18.377289
75%         24.453777      27.877699       17.526962      21.738848
max         37.024049      61.696554       24.984341      70.721654

       contrast_mean3  contrast_var3  contrast_mean4  contrast_var4  \
count      810.000000     810.000000      810.000000     810.000000
mean        19.522501      21.521691       19.202640      16.089759
std          2.605037       5.621524        2.568635       5.574876
min         13.599106      10.259985       13.940119       5.418330
25%         17.666042      17.564770       17.504665      12.312705
50%         19.096981      20.711707       18.784313      15.365789
75%         21.068680      24.816965       20.563879      18.910331
max         29.743913      49.453293       30.059847      46.307186

       contrast_mean5  contrast_var5  contrast_mean6  contrast_var6  \
count      810.000000     810.000000      810.000000     810.000000
mean        19.210271      12.790077       18.692390      10.483000
std          2.368224       5.418710        2.144663       6.414777
min         14.029905       3.300037       14.596196       2.340274
25%         17.589000       9.179146       17.137184       6.297250
50%         18.782413      12.024655       18.203382       8.868925
75%         20.596272      15.504680       19.878121      12.943759
max         28.462142      47.719603       26.927672      51.601413

       contrast_mean7  contrast_var7
count      810.000000     810.000000
mean        28.936525      13.333970
std         11.291930      19.160483
min         14.090684       1.386629
25%         16.514608       4.189467
50%         32.581368       7.584385
75%         39.964493      14.245586
max         46.506639     176.698023

Feature variances:
contrast_mean1     16.548593
contrast_var1      58.404060
contrast_mean2      4.825882
contrast_var2      29.979863
contrast_mean3      6.786218
contrast_var3      31.601531
contrast_mean4      6.597887
contrast_var4      31.079242
contrast_mean5      5.608483
contrast_var5      29.362420
contrast_mean6      4.599580
contrast_var6      41.149367
```

```
contrast_mean7    127.507672
contrast_var7     367.124090
dtype: float64

Class-wise mean for all contrast bands:

--- Band 1 ---
Mean: genre
0    22.323074
1    19.071175
2    24.588116
3    22.789742
4    22.301191
5    20.105663
6    20.716312
7    21.225680
8    22.848393
9    21.529991
Name: contrast_mean1, dtype: float64
Var: genre
0    23.249349
1    20.952272
2    24.081475
3    24.937052
4    26.391534
5    21.649323
6    19.419693
7    27.496600
8    27.837224
9    21.214249
Name: contrast_var1, dtype: float64

--- Band 2 ---
Mean: genre
0    17.226454
1    18.059116
2    17.739168
3    15.040482
4    14.665045
5    17.388479
6    14.254629
7    16.063416
8    16.146493
9    16.071450
Name: contrast_mean2, dtype: float64
Var: genre
0    21.007138
1    20.032298
```

```
2    19.196904
3    17.738306
4    20.010184
5    20.589467
6    14.373978
7    21.991499
8    19.796717
9    17.968668
Name: contrast_var2, dtype: float64

--- Band 3 ---
Mean: genre
0    20.566189
1    22.240446
2    21.391942
3    18.112460
4    17.576234
5    21.840445
6    16.355426
7    19.168039
8    19.248757
9    19.024267
Name: contrast_mean3, dtype: float64
Var: genre
0    23.885526
1    19.098361
2    21.228858
3    19.383257
4    22.301576
5    24.864281
6    15.744375
7    23.878742
8    23.658190
9    19.826298
Name: contrast_var3, dtype: float64

--- Band 4 ---
Mean: genre
0    19.746865
1    23.461793
2    20.660931
3    18.187405
4    16.981679
5    21.772476
6    16.099092
7    18.770962
8    18.470955
9    18.605633
```

```
Name: contrast_mean4, dtype: float64
Var: genre
0    18.499768
1    14.114528
2    16.614157
3    14.348517
4    15.747918
5    22.052117
6     9.661609
7    18.247076
8    15.978410
9    14.249868
Name: contrast_var4, dtype: float64

--- Band 5 ---
Mean: genre
0    20.065594
1    23.444202
2    20.701932
3    18.304042
4    17.135737
5    21.071056
6    16.176880
7    18.663720
8    18.804242
9    18.486756
Name: contrast_mean5, dtype: float64
Var: genre
0    15.140783
1    12.092893
2    14.382238
3    11.639027
4    10.621880
5    18.762291
6     6.192789
7    14.135945
8    12.929706
9    10.759114
Name: contrast_var5, dtype: float64

--- Band 6 ---
Mean: genre
0    19.199012
1    22.443177
2    20.161246
3    17.727721
4    16.891973
5    19.713043
```

```
6    17.317910
7    17.890103
8    18.438720
9    17.958954
Name: contrast_mean6, dtype: float64
Var: genre
0    12.047317
1     9.769668
2    13.273870
3     9.753262
4     7.824614
5    13.310041
6     6.811692
7    12.381970
8    10.989552
9     7.838474
Name: contrast_var6, dtype: float64

--- Band 7 ---
Mean: genre
0    38.070320
1    25.907917
2    27.557955
3    29.309675
4    27.189886
5    26.275126
6    41.476948
7    15.957493
8    26.051772
9    31.427796
Name: contrast_mean7, dtype: float64
Var: genre
0    13.853171
1    15.138418
2    12.029228
3    10.936337
4    17.456421
5     9.325065
6    29.527219
7     4.572389
8    12.902601
9     9.972500
Name: contrast_var7, dtype: float64

Min / Max values:
     contrast_mean1  contrast_var1  contrast_mean2  contrast_var2  \
min       13.499980       7.625196       11.713613       9.969447
max       37.024049      61.696554       24.984341      70.721654
```

```
     contrast_mean3  contrast_var3  contrast_mean4  contrast_var4  \
min       13.599106      10.259985       13.940119       5.418330
max       29.743913      49.453293       30.059847      46.307186

     contrast_mean5  contrast_var5  contrast_mean6  contrast_var6  \
min       14.029905       3.300037       14.596196       2.340274
max       28.462142      47.719603       26.927672      51.601413

     contrast_mean7  contrast_var7
min       14.090684       1.386629
max       46.506639     176.698023
```

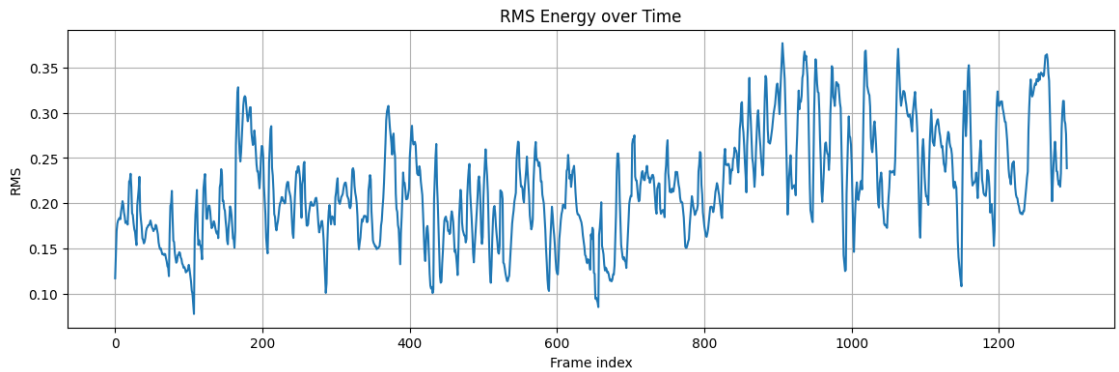
The extraction is correct and working.

4.3 RMS

Root Mean Square (RMS) energy is a time-domain audio feature that measures the average power of an audio signal over short time frames. It is computed by squaring the signal amplitudes, averaging them within each frame, and taking the square root, which produces a stable and physically meaningful estimate of signal intensity. RMS effectively captures the loudness and dynamic behavior of audio, making it useful for distinguishing between music genres with different energy profiles. Its main strengths are robustness to signal fluctuations, low computational cost, and complementarity to spectral features such as MFCCs, as it encodes information about intensity and dynamics that frequency-based features do not capture.

RMS provides different information compared to MFCCs and spectral contrast because it operates in the time domain and focuses exclusively on signal energy rather than frequency content. While MFCCs describe the overall spectral shape and timbral characteristics of audio and spectral contrast captures differences between spectral peaks and valleys, both features are derived from the frequency domain and are largely insensitive to absolute loudness. In contrast, RMS encodes how strong or weak the audio signal is over time, capturing dynamic and intensity-related properties. This makes RMS complementary rather than redundant, as it adds information about energy and temporal variation that is not represented by spectral features, thereby enriching the overall feature set used for classification.

[27]:
```
rms = librosa.feature.rms(y=x)[0]

plt.figure(figsize=(14, 4))
plt.plot(rms)
plt.title("RMS Energy over Time")
plt.xlabel("Frame index")
plt.ylabel("RMS")
plt.grid()
plt.show()
```



Spikes → loud sections (chorus, drops, distortion)

Valleys → silence, intros, breakdowns

Genres: Metal / Rock → high, stable RMS Classical / Jazz → wide RMS dynamics HipHop / Pop → structured RMS patterns

[28]:
```
def extract_rms_features(dataset):
    rms_features = []

    for entry in tqdm(dataset, desc="Extracting RMS Features"):
        audio_array = entry['audio']['array']
        sampling_rate = entry['audio']['sampling_rate']

        # Compute RMS over frames
        rms = librosa.feature.rms(y=audio_array)[0]

        feature_dict = {
            "rms_mean": np.mean(rms),
            "rms_var": np.var(rms),
            "genre": entry["genre"]
        }

        rms_features.append(feature_dict)

    return rms_features
```

Checking that it works:

[29]:
```
# --- Extract RMS features ---
df_rms = pd.DataFrame(extract_rms_features(filtered_dataset))

# 1) Basic checks
print("NaNs:", df_rms.isna().sum().sum())
print("All finite:", np.isfinite(df_rms.drop(columns=["genre"])).all().all())
```

```python
print("Shape:", df_rms.shape)

# 2) Descriptive statistics
print("\nDescriptive stats:")
print(df_rms.drop(columns=["genre"]).describe())

# 3) Variance check (low-variance features)
print("\nFeature variances:")
print(df_rms.drop(columns=["genre"]).var())

# 4) Class-wise mean (quick discriminative sanity check)
print("\nClass-wise RMS mean:")
print(df_rms.groupby("genre")[["rms_mean"]].mean())

# 5) Value range sanity check
print("\nMin / Max values:")
print(df_rms.drop(columns=["genre"]).agg(["min", "max"]))
```

```
Extracting RMS Features:   0%|          | 0/810 [00:00<?, ?it/s]

NaNs: 0
All finite: True
Shape: (810, 3)

Descriptive stats:
         rms_mean     rms_var
count  810.000000  810.000000
mean     0.132396    0.003023
std      0.062007    0.003511
min      0.020713    0.000078
25%      0.090492    0.001000
50%      0.122749    0.001861
75%      0.173566    0.003568
max      0.397734    0.027704

Feature variances:
rms_mean    0.003845
rms_var     0.000012
dtype: float32

Class-wise RMS mean:
       rms_mean
genre
0      0.133839
1      0.052902
2      0.123417
3      0.137688
4      0.178278
```

```
5      0.088350
6      0.155617
7      0.201356
8      0.116846
9      0.124697

Min / Max values:
     rms_mean   rms_var
min  0.020713  0.000078
max  0.397734  0.027704
```

The feature extraction is working properly.

Explore and add other features useful for classification (10 points)

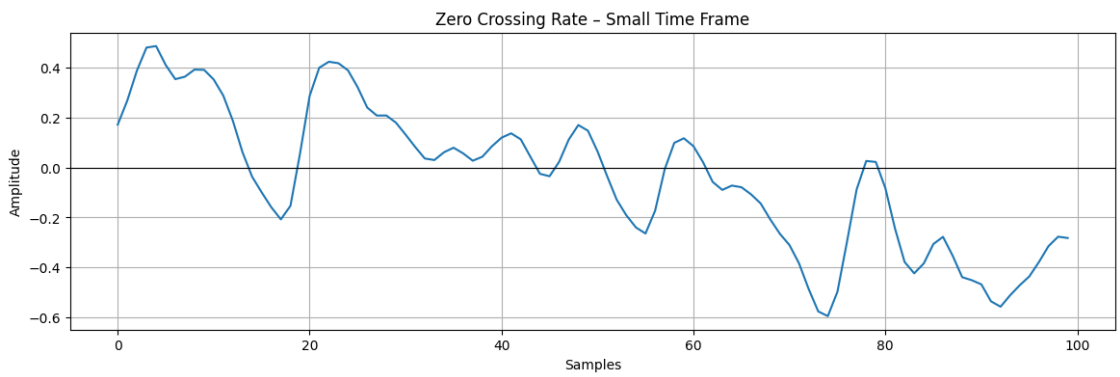
It can be in a single function or in separated functions. Select some features and provide a description for each one. You can choose as many features as you like.

4.4 ZERO CROSSING

Zero Crossing Rate (ZCR) measures how frequently an audio signal changes sign, meaning how often the waveform crosses the zero-amplitude axis from positive to negative or vice versa. Because audio signals naturally oscillate around zero, each crossing reflects a rapid change in the signal's shape. Signals with a lot of high-frequency content, noise, or sharp transients tend to cross zero more often, resulting in a high ZCR, while smoother, more harmonic and tonal signals cross zero less frequently, producing a low ZCR. For this reason, ZCR is commonly used to capture the percussive and noisy characteristics of sound. In music genre classification, rhythm-driven and aggressive genres such as Metal, Hip-Hop, and Disco typically exhibit higher ZCR values due to fast beats and abrupt signal changes, whereas Classical, Jazz, and Blues generally show lower ZCR values because of their smoother, sustained harmonic structures. As a result, ZCR is a useful feature for distinguishing energetic, percussive genres from more melodic and harmonic ones.

[30]:
```
#demonstrative plot

# take one sample from the filtered dataset
entry = filtered_dataset[0]
x = entry["audio"]["array"]
sr = entry["audio"]["sampling_rate"]

# short frame for visualization
m0 = 10000
m1 = 10100

plt.figure(figsize=(14, 4))
plt.plot(x[m0:m1])
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Zero Crossing Rate - Small Time Frame")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
```

```
plt.grid()
plt.show()

# compute zero crossings in the frame
zcr_frame = librosa.zero_crossings(x[m0:m1], pad=False)
print(f"Zero crossings in this frame: {np.sum(zcr_frame)}")
```




```
Zero crossings in this frame: 9
```

[31]:
```
def extract_zcr_features(dataset):
    zcr_features = []

    for entry in tqdm(dataset, desc="Extracting ZCR Features"):
        audio_array = entry["audio"]["array"]

        # Compute ZCR per frame
        zcr = librosa.feature.zero_crossing_rate(y=audio_array)[0]

        feature_dict = {
            "zcr_mean": np.mean(zcr),
            "zcr_var": np.var(zcr),
            "genre": entry["genre"]
        }

        zcr_features.append(feature_dict)

    return pd.DataFrame(zcr_features)
```

Checking if it works

[32]:
```
# ---- RUN TESTS ----
df_zcr = extract_zcr_features(filtered_dataset)

# 1) NaN check
```

```python
print("NaNs:", df_zcr.isna().sum().sum())

# 2) Finite check
print("All finite:", np.isfinite(df_zcr.drop(columns=["genre"])).all().all())

# 3) Shape
print("Shape:", df_zcr.shape)

# 4) Descriptive stats
print("\nDescriptive stats:")
print(df_zcr.drop(columns=["genre"]).describe())

# 5) Feature variance
print("\nFeature variances:")
print(df_zcr.drop(columns=["genre"]).var().astype(np.float32))

# 6) Class-wise mean
print("\nClass-wise ZCR mean:")
print(df_zcr.groupby("genre")[["zcr_mean"]].mean())

# 7) Min/Max values
print("\nMin / Max values:")
print(df_zcr.drop(columns=["genre"]).agg(["min", "max"]))
```

```
Extracting ZCR Features:   0%|          | 0/810 [00:00<?, ?it/s]

NaNs: 0
All finite: True
Shape: (810, 3)

Descriptive stats:
         zcr_mean     zcr_var
count  810.000000  810.000000
mean     0.103583    0.003001
std      0.040932    0.003011
min      0.021697    0.000117
25%      0.071141    0.001094
50%      0.099368    0.002099
75%      0.131751    0.003642
max      0.274631    0.028718

Feature variances:
zcr_mean    0.001675
zcr_var     0.000009
dtype: float32

Class-wise ZCR mean:
       zcr_mean
```

```
genre
0      0.079278
1      0.082920
2      0.083485
3      0.130504
4      0.108178
5      0.075945
6      0.146331
7      0.133076
8      0.088608
9      0.108408

Min / Max values:
     zcr_mean   zcr_var
min  0.021697  0.000117
max  0.274631  0.028718
```

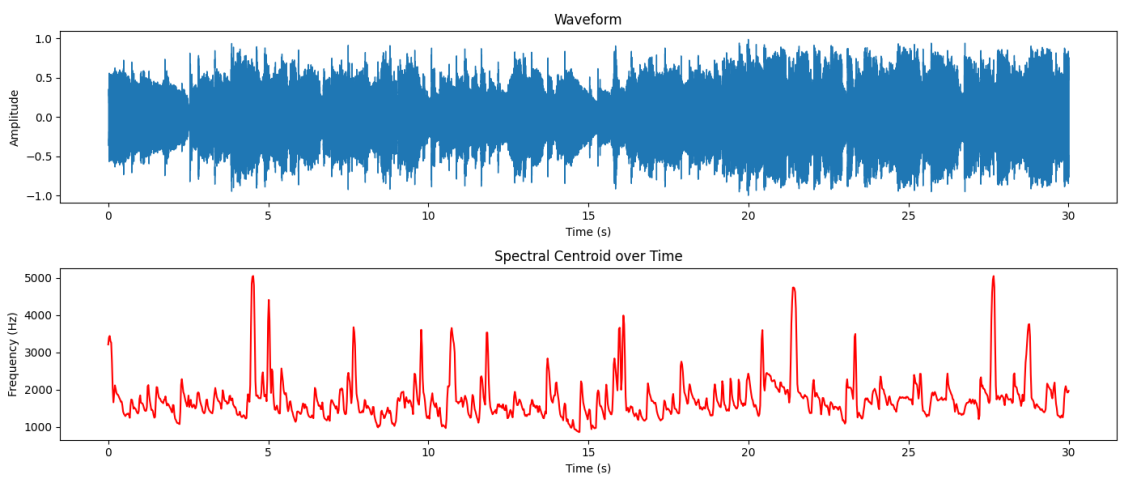
The feature extraction is working properly.

4.5 SPECTRAL CENTROID

Spectral Centroid is a measure that describes the "center of mass" of an audio signal's frequency spectrum, indicating where most of the spectral energy is concentrated. To compute it, the audio is transformed into the frequency domain using the Fast Fourier Transform (FFT), producing a set of frequency bins each with a corresponding magnitude. The centroid is calculated by taking a weighted average of the frequencies, where the weights are the magnitudes of each bin. This means that higher-frequency components with greater energy pull the centroid upward, while lower-frequency energy keeps it lower. As a result, the spectral centroid is closely related to the perceived brightness of a sound: a high centroid corresponds to bright, sharp, high-frequency dominant timbres, while a low centroid corresponds to darker, bass-heavy, low-frequency dominant timbres. In genre classification, this feature helps distinguish between music styles that emphasize high-energy, bright sounds (such as Metal, Rock, and Pop) and those that tend to be warmer and more mellow (such as Blues, Classical, and Jazz). Thus, spectral centroid effectively separates bright versus dark sonic textures across genres.

[33]:
```
#demonstrative plot

entry = filtered_dataset[0]
x = entry["audio"]["array"]
sr = entry["audio"]["sampling_rate"]

# Compute spectral centroid
centroid = librosa.feature.spectral_centroid(y=x, sr=sr)[0]

# Time axis
frames = range(len(centroid))
t = librosa.frames_to_time(frames, sr=sr)
```

```
plt.figure(figsize=(14, 6))

# Waveform
plt.subplot(2, 1, 1)
librosa.display.waveshow(x, sr=sr)
plt.title("Waveform")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")

# Spectral centroid
plt.subplot(2, 1, 2)
plt.plot(t, centroid, color="r")
plt.title("Spectral Centroid over Time")
plt.xlabel("Time (s)")
plt.ylabel("Frequency (Hz)")
plt.tight_layout()
plt.show()
```



Peaks in centroid → moments with more high-frequency content

Flat low curve → bass-dominated or smooth passages

This plot makes it obvious why centroid RMS:

RMS = how strong

Centroid = where in frequency

[34]:
```
def extract_spectral_centroid_features(dataset):
    centroid_features = []

    for entry in tqdm(dataset, desc="Extracting Spectral Centroid"):
        x = entry["audio"]["array"]
```

```
        sr = entry["audio"]["sampling_rate"]

        centroid = librosa.feature.spectral_centroid(y=x, sr=sr)[0]

        feature_dict = {
            "centroid_mean": np.mean(centroid),
            "centroid_var": np.var(centroid),
            "genre": entry["genre"]
        }

        centroid_features.append(feature_dict)

    return pd.DataFrame(centroid_features)
```

Checking that it works

[35]:
```
df_centroid = extract_spectral_centroid_features(filtered_dataset)

# 1) NaN check
print("NaNs:", df_centroid.isna().sum().sum())

# 2) Finite check
print("All finite:", np.isfinite(df_centroid.drop(columns=["genre"])).all().
 ↪all())

# 3) Shape
print("Shape:", df_centroid.shape)

# 4) Descriptive stats
print("\nDescriptive stats:")
print(df_centroid[['centroid_mean', 'centroid_var']].describe())

# 5) Feature variances
print("\nFeature variances:")
print(df_centroid[['centroid_mean', 'centroid_var']].var())

# 6) Class-wise centroid mean
print("\nClass-wise centroid mean:")
print(df_centroid.groupby("genre")["centroid_mean"].mean())

# 7) Min / Max
print("\nMin / Max values:")
print(df_centroid[['centroid_mean', 'centroid_var']].agg(["min", "max"]))
```

```
Extracting Spectral Centroid:   0%|          | 0/810 [00:00<?, ?it/s]

NaNs: 0
All finite: True
```

```
Shape: (810, 3)

Descriptive stats:
       centroid_mean  centroid_var
count     810.000000  8.100000e+02
mean     2223.134623  4.733627e+05
std       695.495109  3.893678e+05
min       570.349904  1.485809e+04
25%      1663.227037  1.946820e+05
50%      2237.053638  3.428611e+05
75%      2698.198596  6.191675e+05
max      4281.984674  2.601903e+06

Feature variances:
centroid_mean    4.837134e+05
centroid_var     1.516073e+11
dtype: float64

Class-wise centroid mean:
genre
0    1739.589506
1    1430.664493
2    1887.719472
3    2652.177497
4    2546.456498
5    1793.407368
6    2597.630861
7    3074.694904
8    2135.173699
9    2248.576288
Name: centroid_mean, dtype: float64

Min / Max values:
     centroid_mean  centroid_var
min     570.349904  1.485809e+04
max    4281.984674  2.601903e+06
```

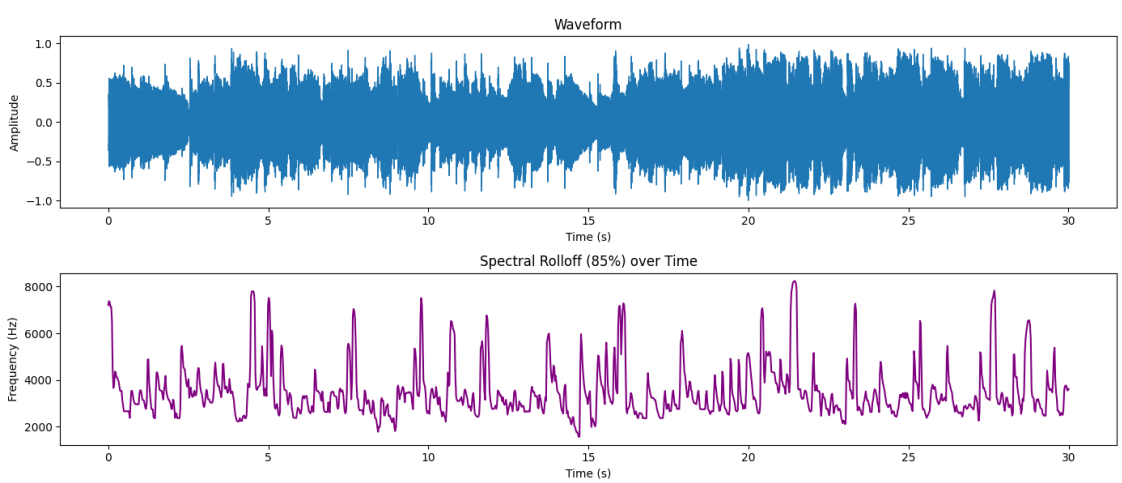
The feature extraction is working properly.

4.6 SPECTRAL ROLLOFF

Spectral Rolloff is a spectral feature that identifies the frequency below which a specified percentage of the total spectral energy is contained. In other words, it marks the cutoff frequency that separates the bulk of the sound's energy from the higher-frequency tail. This makes rolloff a useful indicator of how much high-frequency content a signal contains: a higher rolloff value means that a larger portion of the energy is located in higher frequencies, while a lower rolloff indicates that most energy is concentrated in lower frequencies. Compared to spectral centroid, rolloff is often more robust to noise and outliers because it is based on energy accumulation rather than

an average of all frequency magnitudes. In genre classification, spectral rolloff helps distinguish bright, high-energy genres such as Metal, Hip-Hop, and Disco—which tend to have higher rolloff values—from smoother, more harmonic genres like Classical, Jazz, and Blues, which typically have lower rolloff values. Thus, spectral rolloff provides a reliable measure of timbral brightness and high-frequency emphasis.

[36]:
```
#demonstrative plot

entry = filtered_dataset[0]
x = entry["audio"]["array"]
sr = entry["audio"]["sampling_rate"]

# Spectral rolloff
rolloff = librosa.feature.spectral_rolloff(y=x, sr=sr, roll_percent=0.85)[0]

t = librosa.frames_to_time(range(len(rolloff)), sr=sr)

plt.figure(figsize=(14, 6))

# Waveform
plt.subplot(2, 1, 1)
librosa.display.waveshow(x, sr=sr)
plt.title("Waveform")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")

# Rolloff
plt.subplot(2, 1, 2)
plt.plot(t, rolloff, color="purple")
plt.title("Spectral Rolloff (85%) over Time")
plt.xlabel("Time (s)")
plt.ylabel("Frequency (Hz)")
plt.tight_layout()
plt.show()
```



[37]:
```
def extract_spectral_rolloff_features(dataset, roll_percent=0.85):
    rolloff_features = []

    for entry in tqdm(dataset, desc="Extracting Spectral Rolloff"):
        x = entry["audio"]["array"]
        sr = entry["audio"]["sampling_rate"]

        rolloff = librosa.feature.spectral_rolloff(
            y=x, sr=sr, roll_percent=roll_percent
        )[0]

        feature_dict = {
            "rolloff_mean": np.mean(rolloff),
            "rolloff_var": np.var(rolloff),
            "genre": entry["genre"]
        }

        rolloff_features.append(feature_dict)

    return pd.DataFrame(rolloff_features)
```

Checking that it works

[38]:
```
df_rolloff = extract_spectral_rolloff_features(filtered_dataset)

# 1) NaN check
print("NaNs:", df_rolloff.isna().sum().sum())

# 2) Finite check
print("All finite:", np.isfinite(df_rolloff.drop(columns=["genre"])).all().
 ↪all())

# 3) Shape
print("Shape:", df_rolloff.shape)

# 4) Descriptive stats
print("\nDescriptive stats:")
print(df_rolloff[['rolloff_mean', 'rolloff_var']].describe())

# 5) Feature variances
print("\nFeature variances:")
print(df_rolloff[['rolloff_mean', 'rolloff_var']].var())

# 6) Class-wise rolloff mean
print("\nClass-wise rolloff mean:")
```

```python
print(df_rolloff.groupby("genre")["rolloff_mean"].mean())

# 7) Min / Max
print("\nMin / Max values:")
print(df_rolloff[['rolloff_mean', 'rolloff_var']].agg(["min", "max"]))
```

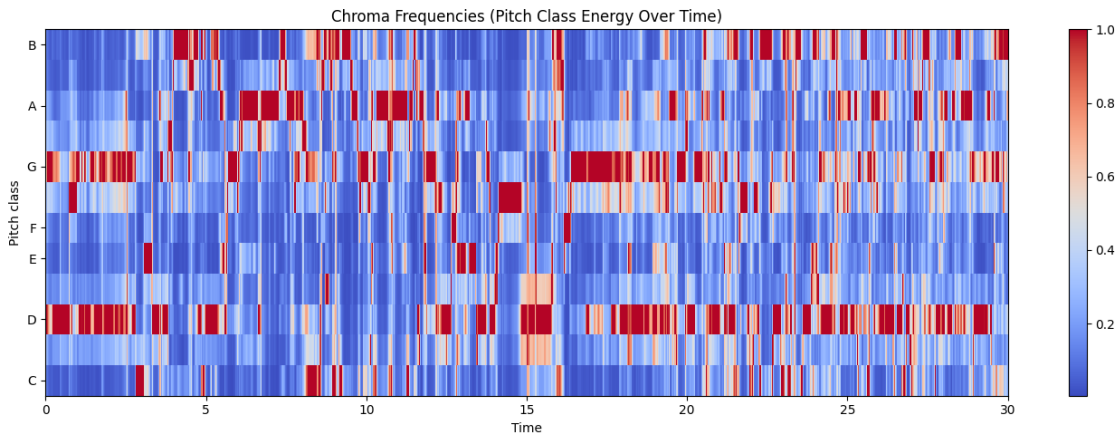
```
Extracting Spectral Rolloff:   0%|          | 0/810 [00:00<?, ?it/s]

NaNs: 0
All finite: True
Shape: (810, 3)

Descriptive stats:
       rolloff_mean   rolloff_var
count    810.000000  8.100000e+02
mean    4640.965749  1.863820e+06
std     1525.127623  1.396224e+06
min      929.308431  7.591046e+04
25%     3504.711999  8.049576e+05
50%     4720.076460  1.507475e+06
75%     5580.207602  2.600606e+06
max     8483.382577  8.308329e+06

Feature variances:
rolloff_mean    2.326014e+06
rolloff_var     1.949442e+12
dtype: float64

Class-wise rolloff mean:
genre
0    3676.405550
1    2653.336587
2    3926.001263
3    5594.939030
4    5402.981901
5    3766.533781
6    5142.846680
7    6657.339786
8    4512.673521
9    4699.634903
Name: rolloff_mean, dtype: float64

Min / Max values:
     rolloff_mean   rolloff_var
min    929.308431  7.591046e+04
max   8483.382577  8.308329e+06
```

The feature extraction is working properly.

4.7 CHROMA FREQUENCIES

Chroma Frequencies (Chroma Features) capture the distribution of pitch classes in an audio signal, summarizing musical content in terms of the twelve semitones of the Western scale. To compute chroma, the audio spectrum is mapped into 12 bins, each corresponding to one of the twelve pitch classes (C, C /D , D, …, B). Energy from all frequencies that belong to the same pitch class is accumulated into the corresponding bin, creating a compact representation of harmonic content that is largely invariant to octave changes. This makes chroma particularly effective at representing chord structures, tonal centers, and harmonic progressions. Signals with clear, stable harmony (e.g., strong chords and consistent tonality) generate pronounced chroma patterns, while percussive or noisy genres tend to produce weaker or less stable chroma profiles. In genre classification, chroma features are useful because they emphasize harmonic consistency: genres such as Classical, Jazz, Blues, and Reggae—where chord progressions and tonal structure are prominent—usually exhibit strong chroma signals, whereas rhythm-driven genres like Metal, Hip-Hop, and Disco typically show weaker chroma patterns. Overall, chroma features help distinguish music based on tonal and harmonic characteristics rather than purely on timbre or rhythm.

[39]:
```
#demonstrative plot

# load one example
entry = filtered_dataset[0]
y = entry["audio"]["array"]
sr = entry["audio"]["sampling_rate"]

# compute chroma
chroma = librosa.feature.chroma_stft(y=y, sr=sr)

plt.figure(figsize=(14, 5))
librosa.display.specshow(
    chroma,
    x_axis="time",
    y_axis="chroma",
    cmap="coolwarm"
)
plt.colorbar()
plt.title("Chroma Frequencies (Pitch Class Energy Over Time)")
plt.tight_layout()
plt.show()
```



Y-axis → pitch classes (C → B)

X-axis → time

Color → energy of that pitch class

Horizontal stable bands → strong harmonic structure

Chaotic patterns → percussive / noisy genres

[40]:
```
def extract_chroma_features(dataset):
    chroma_features = []

    for entry in tqdm(dataset, desc="Extracting Chroma Features"):
        y = entry["audio"]["array"]
        sr = entry["audio"]["sampling_rate"]

        chroma = librosa.feature.chroma_stft(y=y, sr=sr)

        chroma_mean = np.mean(chroma, axis=1)
        chroma_var = np.var(chroma, axis=1)

        feature_dict = {}

        for i in range(12):
            feature_dict[f"chroma_mean{i+1}"] = chroma_mean[i]
            feature_dict[f"chroma_var{i+1}"] = chroma_var[i]

        feature_dict["genre"] = entry["genre"]

        chroma_features.append(feature_dict)

    return pd.DataFrame(chroma_features)
```

Checking that it works

[41]:
```
df_chroma = pd.DataFrame(extract_chroma_features(filtered_dataset))

# 1) NaN check
print("NaNs:", df_chroma.isna().sum().sum())

# 2) Finite check
print("All finite:", np.isfinite(df_chroma.drop(columns=["genre"])).all().all())

# 3) Shape
print("Shape:", df_chroma.shape)

# 4) Descriptive stats
print("\nDescriptive stats:")
print(df_chroma.drop(columns=["genre"]).describe())

# 5) Feature variances
print("\nFeature variances:")
print(df_chroma.drop(columns=["genre"]).var())

# 6) Class-wise mean (example using chroma_mean1)
print("\nClass-wise chroma_mean1:")
print(df_chroma.groupby("genre")["chroma_mean1"].mean())

# 7) Min / Max values
print("\nMin / Max values:")
print(df_chroma.drop(columns=["genre"]).agg(["min", "max"]))
```

```
Extracting Chroma Features:   0%|          | 0/810 [00:00<?, ?it/s]

NaNs: 0
All finite: True
Shape: (810, 25)

Descriptive stats:
       chroma_mean1  chroma_var1  chroma_mean2  chroma_var2  chroma_mean3  \
count    810.000000   810.000000    810.000000   810.000000    810.000000
mean       0.385435     0.085726      0.360824     0.070242      0.393267
std        0.109639     0.028379      0.112321     0.023814      0.115845
min        0.047439     0.005220      0.078052     0.008850      0.101406
25%        0.311002     0.065761      0.278930     0.054096      0.320447
50%        0.379033     0.084170      0.361987     0.068586      0.393967
75%        0.455334     0.104236      0.430787     0.084891      0.470416
max        0.786571     0.189808      0.806371     0.140043      0.773714

       chroma_var3  chroma_mean4  chroma_var4  chroma_mean5  chroma_var5  …  \
count   810.000000    810.000000   810.000000    810.000000   810.000000  …
```

```
mean      0.086101      0.372503      0.074376      0.390849      0.082247  …
std       0.026863      0.113275      0.026814      0.117365      0.025449  …
min       0.017428      0.074952      0.007094      0.123987      0.012459  …
25%       0.067140      0.292484      0.054945      0.307309      0.065241  …
50%       0.084823      0.370238      0.071874      0.384445      0.080435  …
75%       0.103799      0.444898      0.092738      0.471330      0.100779  …
max       0.176978      0.804350      0.153946      0.777271      0.158181  …

       chroma_mean8  chroma_var8  chroma_mean9  chroma_var9  chroma_mean10  \
count    810.000000   810.000000    810.000000   810.000000     810.000000
mean       0.394680     0.087888      0.378445     0.074797       0.400900
std        0.110401     0.027083      0.114312     0.025161       0.114767
min        0.082315     0.021270      0.090704     0.006376       0.124969
25%        0.319008     0.067690      0.300666     0.057725       0.317110
50%        0.398235     0.087102      0.373383     0.072948       0.391021
75%        0.468826     0.107200      0.454947     0.091187       0.479716
max        0.677884     0.164949      0.776868     0.158084       0.775461

       chroma_var10  chroma_mean11  chroma_var11  chroma_mean12  chroma_var12
count    810.000000     810.000000    810.000000     810.000000    810.000000
mean       0.087241       0.376098      0.077010       0.375704      0.076921
std        0.026346       0.109990      0.026777       0.110085      0.025451
min        0.017362       0.098384      0.008336       0.103744      0.013367
25%        0.067387       0.296820      0.058531       0.297230      0.058697
50%        0.086309       0.367336      0.074917       0.373849      0.075293
75%        0.105517       0.446890      0.091421       0.443137      0.092029
max        0.196781       0.802365      0.171622       0.775290      0.182708

[8 rows x 24 columns]

Feature variances:
chroma_mean1     0.012021
chroma_var1      0.000805
chroma_mean2     0.012616
chroma_var2      0.000567
chroma_mean3     0.013420
chroma_var3      0.000722
chroma_mean4     0.012831
chroma_var4      0.000719
chroma_mean5     0.013774
chroma_var5      0.000648
chroma_mean6     0.011761
chroma_var6      0.000843
chroma_mean7     0.012138
chroma_var7      0.000581
chroma_mean8     0.012188
chroma_var8      0.000733
chroma_mean9     0.013067
```

```
chroma_var9      0.000633
chroma_mean10    0.013172
chroma_var10     0.000694
chroma_mean11    0.012098
chroma_var11     0.000717
chroma_mean12    0.012119
chroma_var12     0.000648
dtype: float64

Class-wise chroma_mean1:
genre
0    0.344658
1    0.321193
2    0.321295
3    0.436081
4    0.452582
5    0.315242
6    0.446977
7    0.404613
8    0.418696
9    0.384893
Name: chroma_mean1, dtype: float64

Min / Max values:
     chroma_mean1  chroma_var1  chroma_mean2  chroma_var2  chroma_mean3  \
min      0.047439     0.005220      0.078052     0.008850      0.101406
max      0.786571     0.189808      0.806371     0.140043      0.773714

     chroma_var3  chroma_mean4  chroma_var4  chroma_mean5  chroma_var5  …  \
min     0.017428      0.074952     0.007094      0.123987     0.012459  …
max     0.176978      0.804350     0.153946      0.777271     0.158181  …

     chroma_mean8  chroma_var8  chroma_mean9  chroma_var9  chroma_mean10  \
min      0.082315     0.021270      0.090704     0.006376       0.124969
max      0.677884     0.164949      0.776868     0.158084       0.775461

     chroma_var10  chroma_mean11  chroma_var11  chroma_mean12  chroma_var12
min      0.017362       0.098384      0.008336       0.103744      0.013367
max      0.196781       0.802365      0.171622       0.775290      0.182708

[2 rows x 24 columns]
```

The feature extraction is working properly.

4.8 SPECTRAL BAND ENERGY

Spectral Band Energy quantifies the total amount of energy contained within predefined frequency bands of an audio signal. By dividing the spectrum into several ranges (e.g., bass, low mid, mid,

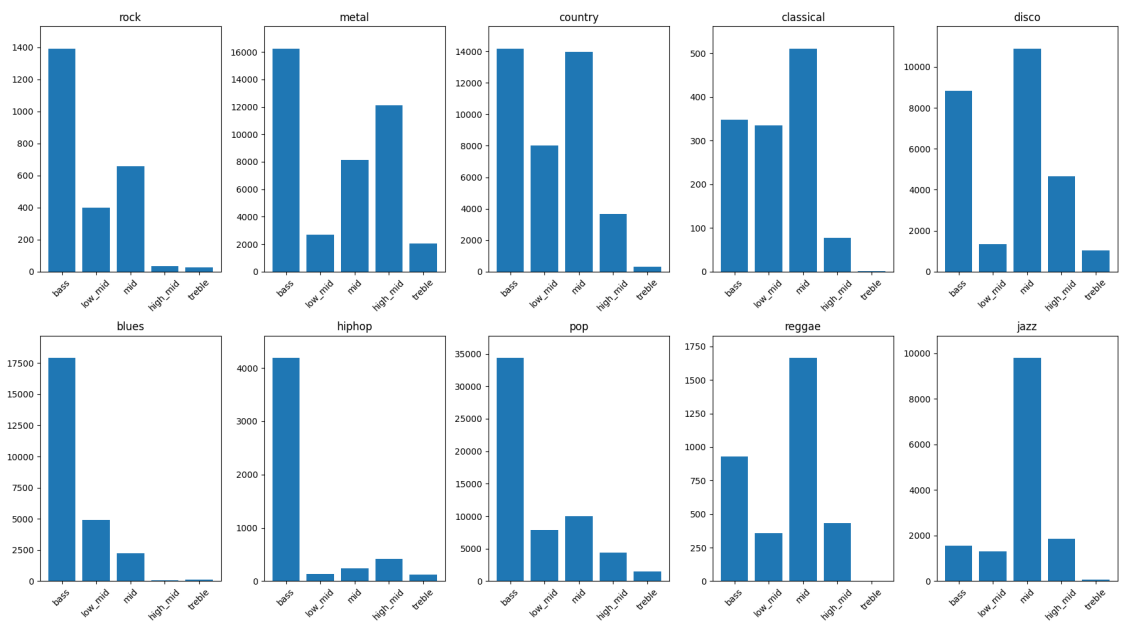
high mid, and treble), this feature provides a compact representation of how energy is distributed across the frequency spectrum. Typical band definitions:

Band Frequency Range Bass 0–250 Hz Low Mid 250–500 Hz Mid 500–2000 Hz High Mid 2000–6000 Hz Treble 6000–12000 Hz

To compute spectral band energy, the audio signal is first transformed into the frequency domain using the Short-Time Fourier Transform (STFT), producing a magnitude spectrum for each time frame. For each predefined band, the magnitudes of all frequency bins within that range are summed to obtain the band's energy in each frame. These values are then averaged across all frames to create a stable feature representing the overall energy distribution in that band. This method captures the tonal balance of a track, making it useful for distinguishing genres: for example, Hip-Hop often shows strong energy in the bass band, Metal frequently exhibits dominant high-mid energy, while Classical music typically displays a more balanced energy distribution across bands. Overall, spectral band energy is effective for identifying genre-specific timbral and frequency-based characteristics.

[42]:
```
#demonstrative plot per genre

def spectral_band_energy(entry, n_fft=2048, hop_length=512):
    audio = entry["audio"]["array"]
    sr = entry["audio"]["sampling_rate"]

    S = np.abs(librosa.stft(audio, n_fft=n_fft, hop_length=hop_length))
    freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)

    bands = {
        "bass": (0, 250),
        "low_mid": (250, 500),
        "mid": (500, 2000),
        "high_mid": (2000, 6000),
        "treble": (6000, 12000)
    }

    band_mean = []
    for low, high in bands.values():
        idx = np.where((freqs >= low) & (freqs < high))[0]
        energy = S[idx, :]**2
        band_mean.append(energy.sum(axis=0).mean())

    return bands.keys(), band_mean


genre_to_entry = {}

for entry in filtered_dataset:
    genre = entry["genre"]
    if genre not in genre_to_entry:
        genre_to_entry[genre] = entry
```

```
# Optional: shuffle
genre_to_entry = dict(random.sample(list(genre_to_entry.items()),␣
 ↪len(genre_to_entry)))


plt.figure(figsize=(18, 10))

for i, (genre_id, entry) in enumerate(genre_to_entry.items()):
    bands, energy = spectral_band_energy(entry)

    plt.subplot(2, 5, i + 1)
    plt.bar(bands, energy)
    plt.title(dataset.features['genre'].int2str(genre_id))
    plt.xticks(rotation=45)
    plt.ylim(0, max(energy) * 1.1)

plt.tight_layout()
plt.show()
```



[43]:
```
def extract_spectral_band_energy(dataset, n_fft=2048, hop_length=512):
    features = []

    for entry in tqdm(dataset, desc="Extracting Spectral Band Energy"):
        audio = entry["audio"]["array"]
        sr = entry["audio"]["sampling_rate"]
```

```
        # STFT magnitude
        S = np.abs(librosa.stft(audio, n_fft=n_fft, hop_length=hop_length))
        freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)

        # Frequency bands
        bands = {
            "bass": (0, 250),
            "low_mid": (250, 500),
            "mid": (500, 2000),
            "high_mid": (2000, 6000),
            "treble": (6000, 12000)
        }

        feature_dict = {}
        for name, (low, high) in bands.items():
            idx = np.where((freqs >= low) & (freqs < high))[0]
            energy = (S[idx, :] ** 2).sum(axis=0)

            # store mean and variance
            feature_dict[f"{name}_energy_mean"] = energy.mean()
            feature_dict[f"{name}_energy_var"] = energy.var()

        feature_dict["genre"] = entry["genre"]
        features.append(feature_dict)

    return pd.DataFrame(features)
```

Checking that it works

[44]:
```
df_band = extract_spectral_band_energy(filtered_dataset)

# 1) NaN check
print("NaNs:", df_band.isna().sum().sum())

# 2) Finite check
print("All finite:", np.isfinite(df_band.drop(columns=["genre"])).all().all())

# 3) Shape
print("Shape:", df_band.shape)

# 4) Descriptive stats
print("\nDescriptive stats:")
print(df_band.drop(columns=["genre"]).describe())

# 5) Feature variances
print("\nFeature variances:")
print(df_band.drop(columns=["genre"]).var())
```

```python
# 6) Class-wise mean for one band (example: bass)
print("\nClass-wise bass energy mean:")
print(df_band.groupby("genre")["bass_energy_mean"].mean())

# 7) Min / Max values
print("\nMin / Max values:")
print(df_band.drop(columns=["genre"]).agg(["min", "max"]))
```

```
Extracting Spectral Band Energy:   0%|          | 0/810 [00:00<?, ?it/s]

NaNs: 0
All finite: True
Shape: (810, 11)

Descriptive stats:
       bass_energy_mean  bass_energy_var  low_mid_energy_mean  \
count        810.000000     8.100000e+02           810.000000
mean       10889.549693     3.525042e+08          2277.844138
std        12213.133290     8.999343e+08          2054.196946
min           38.754182     2.520901e+03            66.995020
25%         2889.390574     8.813541e+06           810.334809
50%         6631.073840     5.188047e+07          1744.393165
75%        14923.911587     2.441477e+08          3135.285191
max        85417.629052     7.729066e+09         19625.610482

       low_mid_energy_var  mid_energy_mean  mid_energy_var  \
count        8.100000e+02       810.000000    8.100000e+02
mean         1.360683e+07      4034.684769    4.479924e+07
std          3.435492e+07      3907.152666    1.081091e+08
min          1.350466e+04        66.526017    6.821651e+03
25%          1.007998e+06      1420.215634    3.534023e+06
50%          4.285323e+06      2887.453563    1.472820e+07
75%          1.377856e+07      5333.002364    4.327491e+07
max          6.838491e+08     38461.756401    1.723311e+09

       high_mid_energy_mean  high_mid_energy_var  treble_energy_mean  \
count            810.000000         8.100000e+02          810.000000
mean            1634.208035         6.668387e+06          359.090670
std             1876.042261         1.391655e+07          589.332322
min                1.780497         2.440537e+01            0.014852
25%              329.748870         2.428085e+05           27.832030
50%             1031.652115         1.629714e+06          117.397656
75%             2167.588676         6.622823e+06          382.889672
max            12621.123683         1.616316e+08         3719.443023

       treble_energy_var
count       8.100000e+02
```

```
mean        1.279622e+06
std         4.511772e+06
min         7.595228e-04
25%         3.277409e+03
50%         4.822455e+04
75%         3.671199e+05
max         7.326334e+07

Feature variances:
bass_energy_mean        1.491606e+08
bass_energy_var         8.098818e+17
low_mid_energy_mean     4.219725e+06
low_mid_energy_var      1.180261e+15
mid_energy_mean         1.526584e+07
mid_energy_var          1.168758e+16
high_mid_energy_mean    3.519535e+06
high_mid_energy_var     1.936704e+14
treble_energy_mean      3.473126e+05
treble_energy_var       2.035608e+13
dtype: float64

Class-wise bass energy mean:
genre
0     8984.369552
1      905.207257
2     6816.014745
3    10150.660253
4    24613.889405
5     3472.545776
6    12880.782317
7    23415.602497
8     9726.109348
9     7399.882136
Name: bass_energy_mean, dtype: float64

Min / Max values:
     bass_energy_mean  bass_energy_var  low_mid_energy_mean  \
min         38.754182     2.520901e+03            66.995020
max      85417.629052     7.729066e+09         19625.610482

     low_mid_energy_var  mid_energy_mean  mid_energy_var  \
min        1.350466e+04        66.526017    6.821651e+03
max        6.838491e+08     38461.756401    1.723311e+09

     high_mid_energy_mean  high_mid_energy_var  treble_energy_mean  \
min              1.780497         2.440537e+01            0.014852
max          12621.123683         1.616316e+08         3719.443023
```

```
     treble_energy_var
min       7.595228e-04
max       7.326334e+07
```

The feature extraction is working properly.

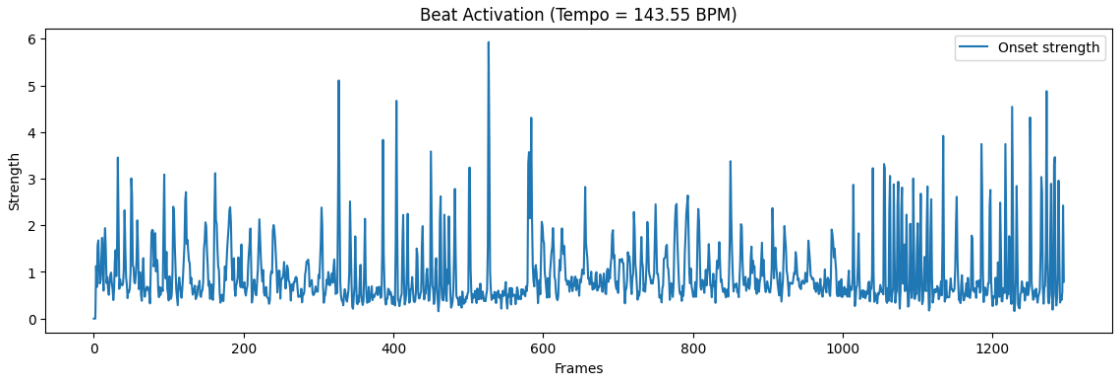
(ADV) Analyze feature importance (5 points)

Analyze the importance of each feature used in the model to understand which variables have the greatest impact on predictions.

4.9 TEMPO AND BEAT FEATURES

Tempo and Beat Features capture the rhythmic structure of a track by estimating its speed and periodicity. Tempo, measured in beats per minute (BPM), is determined by analyzing the time-domain rhythm pattern of the audio signal rather than its frequency content. Librosa estimates tempo by first detecting onsets—moments of sudden energy increase that typically correspond to musical beats—and then constructing an onset envelope that represents rhythmic intensity over time. The algorithm then analyzes this envelope using autocorrelation or Fourier-based methods to identify the dominant periodicity, which is converted into a BPM value. In addition to tempo, beat features often include measures of beat strength or beat confidence, which reflect how clearly the rhythm is expressed. Unlike spectral features, which describe timbre and frequency distribution, tempo and beat features characterize the temporal structure of music. This makes them particularly useful for distinguishing genres with distinct rhythmic profiles: for example, Disco, Hip-Hop, and Rock tend to exhibit high tempo and strong, regular beats; Classical and Jazz often have lower tempo and less consistent beat patterns; Reggae is defined by its steady tempo and distinctive off-beat rhythm; and Metal typically features very high tempo with intense beat energy.

[45]:
```python
#demonstrative plot on a random song
entry = filtered_dataset[123]
y = entry["audio"]["array"]
sr = entry["audio"]["sampling_rate"]

# compute onset envelope
onset_env = librosa.onset.onset_strength(y=y, sr=sr)

# tempo + beats
tempo, beats = librosa.beat.beat_track(onset_envelope=onset_env, sr=sr)

# tempo can be an array -> take first value
tempo_val = tempo[0] if isinstance(tempo, (np.ndarray, list)) else tempo

plt.figure(figsize=(14, 4))
plt.plot(onset_env, label="Onset strength")
plt.title(f"Beat Activation (Tempo = {tempo_val:.2f} BPM)")
plt.xlabel("Frames")
plt.ylabel("Strength")
plt.legend()
plt.show()
```



[46]:
```
def extract_tempo_features(dataset):
    tempo_features = []

    for entry in tqdm(dataset, desc="Extracting Tempo Features"):
        audio = entry["audio"]["array"]
        sr = entry["audio"]["sampling_rate"]

        # 1) Tempo (scalar)
        tempo = librosa.beat.tempo(y=audio, sr=sr)
        tempo_value = float(tempo[0]) if len(tempo) > 0 else np.nan

        # 2) Beat frames (to compute intervals)
        tempo_beats = librosa.beat.beat_track(y=audio, sr=sr)[1]
        beat_times = librosa.frames_to_time(tempo_beats, sr=sr)

        beat_count = int(len(beat_times))

        # intervals
        if beat_count > 1:
            intervals = np.diff(beat_times)
            interval_mean = float(np.mean(intervals))
            interval_var = float(np.var(intervals))
        else:
            interval_mean = np.nan
            interval_var = np.nan

        tempo_features.append({
            "tempo_bpm": tempo_value,
            "beat_count": beat_count,
            "beat_interval_mean": interval_mean,
            "beat_interval_var": interval_var,
            "genre": entry["genre"]
        })
```

```
    return pd.DataFrame(tempo_features)
```

Checking that it works

[47]:
```
df_tempo = extract_tempo_features(filtered_dataset)

# 1) NaN check
print("NaNs:", df_tempo.isna().sum().sum())

# 2) Finite check
print("All finite:", np.isfinite(df_tempo.drop(columns=["genre"])).all().all())

# 3) Shape
print("Shape:", df_tempo.shape)

# 4) Descriptive stats
print("\nDescriptive stats:")
print(df_tempo.drop(columns=["genre"]).describe())

# 5) Feature variances
print("\nFeature variances:")
print(df_tempo.drop(columns=["genre"]).var())

# 6) Class-wise tempo mean
print("\nClass-wise tempo mean:")
print(df_tempo.groupby("genre")["tempo_bpm"].mean())

# 7) Min / Max values
print("\nMin / Max values:")
print(df_tempo.drop(columns=["genre"]).agg(["min", "max"]))
```

```
Extracting Tempo Features:   0%|          | 0/810 [00:00<?, ?it/s]

/tmp/ipython-input-4148432777.py:9: FutureWarning: librosa.beat.tempo
        This function was moved to 'librosa.feature.rhythm.tempo' in librosa
version 0.10.0.
        This alias will be removed in librosa version 1.0.
  tempo = librosa.beat.tempo(y=audio, sr=sr)

NaNs: 0
All finite: True
Shape: (810, 5)

Descriptive stats:
        tempo_bpm  beat_count  beat_interval_mean  beat_interval_var
count  810.000000   810.00000          810.000000         810.000000
mean   121.377192    57.11358            0.529515           0.000477
std     21.268379    14.42210            0.127932           0.000635
```

```
min     69.837416    13.00000            0.255420           0.000008
25%    103.359375    48.00000            0.440915           0.000150
50%    117.453835    56.00000            0.513305           0.000241
75%    135.999178    66.00000            0.603044           0.000552
max    184.570312   117.00000            1.089480           0.006703

Feature variances:
tempo_bpm             4.523439e+02
beat_count            2.079970e+02
beat_interval_mean    1.636665e-02
beat_interval_var     4.033176e-07
dtype: float64

Class-wise tempo mean:
genre
0    125.101981
1    127.515389
2    121.940597
3    120.492938
4    109.759227
5    118.737619
6    122.345533
7    114.057932
8    133.883355
9    119.852355
Name: tempo_bpm, dtype: float64

Min / Max values:
      tempo_bpm  beat_count  beat_interval_mean  beat_interval_var
min   69.837416          13             0.25542           0.000008
max  184.570312         117             1.08948           0.006703
```

(ADV) Analyze correlation of the features (5 points)

Plot correlation diagram and conclude which features are too much correlated and could be removed.

[ ]:
```
```

5 Train a classifier

[48]:
```
# 1) MFCC (n_mfcc=20)
mfcc_df = pd.DataFrame(dfs_mfcc[20])

# 2) Spectral Contrast
contrast_df = df_spcon

# 3) RMS
rms_df = df_rms
```

```
# 4) ZCR
zcr_df =df_zcr

# 5) Spectral Centroid
centroid_df = df_centroid

# 6) Spectral Rolloff
rolloff_df = df_rolloff

# 7) Spectral Band Energy (keep only once)
band_energy_df = df_band

# 8) Chroma
chroma_df = df_chroma

# 9) Tempo features
tempo_df = df_tempo
```

[62]:
```
# 1) Merge all feature dataframes
df_all = pd.concat([
    mfcc_df,
    contrast_df,
    rms_df,
    zcr_df,
    centroid_df,
    rolloff_df,
    band_energy_df,
    chroma_df,
    tempo_df
], axis=1)

# 2) Avoid duplicate genre columns
if "genre" in df_all.columns:
    df_all = df_all.loc[:, ~df_all.columns.duplicated()]

# 3) Remove genre from correlation
X_all = df_all.drop(columns=["genre"])

# 4) Correlation matrix
corr_matrix = X_all.corr()

# 5) Print correlation matrix
print("Correlation Matrix:\n")
print(corr_matrix)

# 6) Find highly correlated pairs
```

```python
threshold = 0.85
high_corr_pairs = []

for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        if abs(corr_matrix.iloc[i, j]) > threshold:
            high_corr_pairs.append(
                (corr_matrix.columns[i], corr_matrix.columns[j], corr_matrix.
 ↪iloc[i, j])
            )

print("\nHighly correlated pairs (|corr| > 0.85):")
for pair in high_corr_pairs:
    print(pair)

# 7) Features to consider removing
to_remove = set([pair[1] for pair in high_corr_pairs])
print("\nFeatures to consider removing:", to_remove)
```

```
Correlation Matrix:

                    mfcc_mean1  mfcc_var1  mfcc_mean2  mfcc_var2  mfcc_mean3  \
mfcc_mean1            1.000000  -0.432696   -0.619152  -0.227551   -0.063677
mfcc_var1            -0.432696   1.000000    0.166154   0.574434    0.198473
mfcc_mean2           -0.619152   0.166154    1.000000  -0.018239   -0.326024
mfcc_var2            -0.227551   0.574434   -0.018239   1.000000    0.316295
mfcc_mean3           -0.063677   0.198473   -0.326024   0.316295    1.000000
…                          …          …           …          …           …
chroma_var12         -0.100729   0.141884    0.001712   0.203658    0.140636
tempo_bpm            -0.091330   0.004765    0.098335  -0.025426   -0.023615
beat_count            0.009658  -0.076056   -0.009275  -0.096336   -0.083253
beat_interval_mean   -0.036945   0.064436    0.041382   0.095434    0.109266
beat_interval_var    -0.280422   0.052710    0.176556   0.029548    0.051820

                    mfcc_var3  mfcc_mean4  mfcc_var4  mfcc_mean5  mfcc_var5  \
mfcc_mean1          -0.268809    0.010108  -0.057349   -0.078500  -0.201872
mfcc_var1            0.597162   -0.186107   0.445659    0.258089   0.488142
mfcc_mean2          -0.009744    0.117420  -0.185467   -0.185562   0.026066
mfcc_var2            0.545691   -0.274370   0.609181    0.332899   0.560369
mfcc_mean3           0.289432   -0.396734   0.315125    0.599130   0.182916
…                          …           …          …           …          …
chroma_var12         0.239617   -0.023570   0.276768    0.142561   0.244840
tempo_bpm           -0.066937    0.012884  -0.051270    0.003192   0.016548
beat_count          -0.053323    0.049030  -0.048764   -0.062874  -0.052484
beat_interval_mean   0.055458   -0.024592   0.068376    0.071423   0.095287
beat_interval_var    0.041837    0.022141  -0.043018    0.034097  -0.022481
```

```
                    …  chroma_mean10  chroma_var10  chroma_mean11  \
mfcc_mean1          …       0.396916     -0.180031       0.392703
mfcc_var1           …      -0.146817      0.092728      -0.077149
mfcc_mean2          …      -0.331970      0.096016      -0.377528
mfcc_var2           …      -0.077197      0.115435      -0.023194
mfcc_mean3          …      -0.081652      0.055096       0.027950
…                 …            …            …             …
chroma_var12        …      -0.063532      0.154148      -0.065547
tempo_bpm           …      -0.000731      0.049147      -0.059501
beat_count          …       0.085746      0.016421       0.048865
beat_interval_mean  …      -0.099661     -0.000547      -0.065882
beat_interval_var   …      -0.214150      0.031793      -0.168040

                    chroma_var11  chroma_mean12  chroma_var12  tempo_bpm  \
mfcc_mean1             -0.146991       0.397743     -0.100729  -0.091330
mfcc_var1               0.140020      -0.067370      0.141884   0.004765
mfcc_mean2              0.039865      -0.392701      0.001712   0.098335
mfcc_var2               0.105764       0.034789      0.203658  -0.025426
mfcc_mean3              0.136072       0.043664      0.140636  -0.023615
…                          …             …            …         …
chroma_var12           -0.179792       0.491448      1.000000  -0.016475
tempo_bpm              -0.079483      -0.037648     -0.016475   1.000000
beat_count             -0.015752      -0.010909     -0.088984   0.449030
beat_interval_mean      0.019197      -0.022763      0.079296  -0.413938
beat_interval_var       0.053606      -0.125901      0.083063  -0.088422

                    beat_count  beat_interval_mean  beat_interval_var
mfcc_mean1            0.009658           -0.036945          -0.280422
mfcc_var1            -0.076056            0.064436           0.052710
mfcc_mean2           -0.009275            0.041382           0.176556
mfcc_var2            -0.096336            0.095434           0.029548
mfcc_mean3           -0.083253            0.109266           0.051820
…                        …                  …                 …
chroma_var12         -0.088984            0.079296           0.083063
tempo_bpm             0.449030           -0.413938          -0.088422
beat_count            1.000000           -0.905266          -0.375797
beat_interval_mean   -0.905266            1.000000           0.483576
beat_interval_var    -0.375797            0.483576           1.000000

[100 rows x 100 columns]

Highly correlated pairs (|corr| > 0.85):
('mfcc_mean2', 'centroid_mean', np.float64(-0.9405910826224079))
('mfcc_mean2', 'rolloff_mean', np.float64(-0.9341232975218681))
('mfcc_var19', 'mfcc_var20', np.float64(0.8737961464849375))
('contrast_mean2', 'contrast_mean3', np.float64(0.8554113913533369))
('contrast_mean3', 'contrast_mean4', np.float64(0.8921427882832222))
('contrast_mean4', 'contrast_mean5', np.float64(0.9184838809275626))
```

```
('rms_mean', 'bass_energy_mean', np.float64(0.8636815631350181))
('rms_var', 'bass_energy_var', np.float64(0.8963433889175172))
('zcr_mean', 'centroid_mean', np.float64(0.8715513350976626))
('centroid_mean', 'rolloff_mean', np.float64(0.978917612700155))
('beat_count', 'beat_interval_mean', np.float64(-0.9052662134211928))

Features to consider removing: {'beat_interval_mean', 'mfcc_var20',
'bass_energy_mean', 'contrast_mean3', 'rolloff_mean', 'contrast_mean5',
'centroid_mean', 'bass_energy_var', 'contrast_mean4'}
```

The correlation matrix reveals several highly correlated feature pairs ($|\text{corr}| > 0.85$), such as centroid_mean and rolloff_mean, mfcc_var19 and mfcc_var20, and multiple spectral contrast bands. These features are candidates for removal because they carry redundant information. However, in this project we will keep all features for now, since the later automatic feature selection step will identify the most relevant subset for each model (LR and SVC) in a more objective and performance-driven way.

5.1 LOGISTIC REGRESSION

5.2 SETTING UP THE PIPELINE

Note that it is at this point that I execute the oversampling technique to balance out the genre unbalances.

The dataset used for genre classification is imbalanced, meaning some genres have significantly fewer samples than others. Training a classifier on such an imbalanced dataset typically results in a model that is biased toward the majority classes, because the learning algorithm minimizes overall error and therefore tends to favor the classes with more examples. To address this, oversampling was applied to the training set. Oversampling increases the number of samples in minority classes so that all classes contribute equally to the training process. Specifically, SMOTE (Synthetic Minority Over-sampling Technique) was used. SMOTE generates new synthetic samples rather than simply duplicating existing ones. For each minority-class sample, SMOTE identifies its k nearest neighbors in feature space and creates new examples by interpolating between the sample and its neighbors. This approach increases class balance while also improving the diversity of the minority class representation, which helps the classifier learn more generalizable decision boundaries. Importantly, oversampling is performed only on the training set and not on the test set. This ensures that the test data remains a realistic representation of the original distribution, avoiding data leakage and providing a valid evaluation of the model's performance. After applying SMOTE, all genres had the same number of samples, confirming that the training set was fully balanced and that the classifier could be trained without bias toward majority genres.

[49]:
```python
# --------- 0) helper: audio hash ----------
def audio_hash(arr):
    a = np.ascontiguousarray(arr)
    return hashlib.sha1(a.tobytes()).hexdigest()

# ----------------------------
# 2) Drop genre from all except mfcc_df
# ----------------------------
```

```python
def drop_genre(df):
    return df.drop(columns=["genre"], errors="ignore")

contrast_df    = drop_genre(contrast_df)
rms_df         = drop_genre(rms_df)
zcr_df         = drop_genre(zcr_df)
centroid_df    = drop_genre(centroid_df)
rolloff_df     = drop_genre(rolloff_df)
band_energy_df = drop_genre(band_energy_df)
chroma_df      = drop_genre(chroma_df)
tempo_df       = drop_genre(tempo_df)

# ----------------------------
# 3) Merge all features
# ----------------------------
df_all = mfcc_df.copy()
for df in [contrast_df, rms_df, zcr_df, centroid_df, rolloff_df,␣
 ↪band_energy_df, chroma_df, tempo_df]:
    df_all = df_all.merge(df, left_index=True, right_index=True)

# ----------------------------
# 4) Add genre once
# ----------------------------
df_all["genre"] = filtered_dataset["genre"]

# ----------------------------
# 5) Fix tempo type
# ----------------------------
df_all["tempo_bpm"] = df_all["tempo_bpm"].apply(
    lambda x: x[0] if isinstance(x, (list, np.ndarray)) else x
)

df_all["genre"] = df_all["genre"].astype(str)


# ----------------------------
# 6) DUPLICATE CLEANING (STRICT)
# ----------------------------
n_before = len(df_all)

X = df_all.drop(columns=["genre"])
y = df_all["genre"].copy()

to_drop = []
conflict_groups = []
dupe_groups = []
```

```python
dups = X[X.duplicated(keep=False)]

if not dups.empty:
    dup_grouped = dups.groupby(list(X.columns)).apply(lambda g: g.index.
 ↪tolist())
    dupe_groups = [g for g in dup_grouped.values]

    for grp in dupe_groups:
        idxs = list(grp)
        raw_genres = [filtered_dataset[i]["genre"] for i in idxs]

        if len(set(raw_genres)) == 1:
            to_drop.extend(idxs[1:])
        else:
            conflict_groups.append(idxs)
            to_drop.extend(idxs)

    df_clean = df_all.drop(index=to_drop, errors="ignore").
 ↪reset_index(drop=True)
else:
    df_clean = df_all.copy()

n_after = len(df_clean)

print("Duplicate cleaning (strict)")
print(f"Before: {n_before}")
print(f"After:  {n_after}")
print(f"Dropped: {n_before - n_after}")
print(f"Duplicate groups found: {len(dupe_groups)}")
print(f"Conflicting-label groups dropped entirely: {len(conflict_groups)}")

if conflict_groups:
    print("Conflicting groups (indices):", conflict_groups)


# ----------------------------
# 7) Prepare X and y from cleaned df
# ----------------------------
X_clean = df_clean.drop(columns=["genre"])
y_clean = df_clean["genre"].copy()

# ----------------------------
# 8) Expand list columns (if any)
# ----------------------------
def expand_list_columns(df):
    new_df = df.copy()
```

```
    list_cols = [c for c in new_df.columns if new_df[c].apply(lambda x:␣
↪isinstance(x, (list, np.ndarray))).any()]

    for col in list_cols:
        expanded = pd.DataFrame(new_df[col].tolist()).add_prefix(col + "_")
        new_df = new_df.drop(columns=[col]).reset_index(drop=True)
        new_df = pd.concat([new_df, expanded], axis=1)

    return new_df

X_clean = expand_list_columns(X_clean)


# ----------------------------
# 9) TRAIN/TEST SPLIT
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X_clean, y_clean,
    test_size=0.2,
    random_state=42,
    stratify=y_clean
)


# ----------------------------
# 10) OVERSAMPLING (ONLY TRAIN)
# ----------------------------
smote = SMOTE(random_state=42)
X_train_os, y_train_os = smote.fit_resample(X_train, y_train)


# ----------------------------
# 11) CHECK BALANCE
# ----------------------------
print("Before oversampling (train):")
print(y_train.value_counts().sort_index())

print("\nAfter oversampling (train):")
print(y_train_os.value_counts().sort_index())
```

```
Duplicate cleaning (strict)
Before: 810
After:  810
Dropped: 0
Duplicate groups found: 0
Conflicting-label groups dropped entirely: 0
Before oversampling (train):
```

```
genre
0    67
1    47
2    66
3    71
4    62
5    71
6    61
7    64
8    72
9    67
Name: count, dtype: int64

After oversampling (train):
genre
0    72
1    72
2    72
3    72
4    72
5    72
6    72
7    72
8    72
9    72
Name: count, dtype: int64
```

[50]:

```
# ----------------------------
# 12) Map numeric labels back to genre names (if available)
# ----------------------------
try:
    genre_names = filtered_dataset.features["genre"].names
except Exception:
    genre_names = None

if genre_names is not None:
    def id_to_name(v):
        try:
            idx = int(v)
            return genre_names[idx]
        except Exception:
            return v

    y_named = y_clean.map(id_to_name)
else:
    y_named = y_clean.copy()
```

```python
print("Label legend:")
if genre_names is not None:
    for i, name in enumerate(genre_names):
        print(f"{i} -> {name}")
else:
    for i, name in enumerate(sorted(set(y_named))):
        print(f"{i} -> {name}")

# ----------------------------
# 13) Encode labels (names -> integers)
# ----------------------------
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y_named)
print("Encoder classes (order):", label_encoder.classes_)
```

```
Label legend:
0 -> blues
1 -> classical
2 -> country
3 -> disco
4 -> hiphop
5 -> jazz
6 -> metal
7 -> pop
8 -> reggae
9 -> rock
Encoder classes (order): ['blues' 'classical' 'country' 'disco' 'hiphop' 'jazz'
'metal' 'pop'
 'reggae' 'rock']
```

[51]:
```python
# ----------------------------
# 14) Train-test split (same split for all tests)
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X_clean,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

# ----------------------------
# 15) Pipeline + Logistic Regression (baseline)
# ----------------------------
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
```

```python
    ('classifier', LogisticRegression(C=0.1, max_iter=5000, random_state=42))
])

pipeline.fit(X_train, y_train)

# ----------------------------
# 16) Predict & Evaluate (baseline)
# ----------------------------
y_pred = pipeline.predict(X_test)
train_pred = pipeline.predict(X_train)


accuracy = accuracy_score(y_test, y_pred)
print("Logistic Regression Accuracy:", accuracy)
print("Train acc:", accuracy_score(y_train, train_pred))


print("\nClassification Report:\n")
print(classification_report(
    y_test,
    y_pred,
    target_names=label_encoder.classes_
))

print("\nConfusion Matrix:\n")
cm = confusion_matrix(y_test, y_pred)

cm_df = pd.DataFrame(
    cm,
    index=label_encoder.classes_,
    columns=label_encoder.classes_
)

print(cm_df)
```

```
Logistic Regression Accuracy: 0.7654320987654321
Train acc: 0.9120370370370371

Classification Report:

              precision    recall  f1-score   support

       blues       0.71      0.59      0.65        17
   classical       1.00      0.92      0.96        12
     country       0.63      0.71      0.67        17
       disco       0.67      0.71      0.69        17
      hiphop       0.85      0.73      0.79        15
```

```
        jazz       0.81      0.94      0.87        18
       metal       0.88      0.93      0.90        15
         pop       0.93      0.88      0.90        16
      reggae       0.78      0.78      0.78        18
        rock       0.53      0.53      0.53        17

    accuracy                           0.77       162
   macro avg       0.78      0.77      0.77       162
weighted avg       0.77      0.77      0.76       162


Confusion Matrix:

           blues  classical  country  disco  hiphop  jazz  metal  pop  reggae  \
blues         10          0        0      0       0     1      0    0       1
classical      0         11        0      0       0     1      0    0       0
country        3          0       12      1       0     0      0    0       0
disco          0          0        2     12       1     0      0    0       0
hiphop         0          0        0      1      11     0      1    1       1
jazz           0          0        0      0       0    17      0    0       1
metal          0          0        0      1       0     0     14    0       0
pop            0          0        2      0       0     0      0   14       0
reggae         1          0        1      0       1     1      0    0      14
rock           0          0        2      3       0     1      1    0       1

           rock
blues         5
classical     0
country       1
disco         2
hiphop        0
jazz          0
metal         0
pop           0
reggae        0
rock          9
```

The current model shows signs of overfitting, suggesting that the feature set may contain redundant or non-informative variables. To improve generalization, the feature set is reduced by evaluating different combinations of feature groups (MFCC, spectral contrast, chroma, RMS, ZCR, etc.). For each combination, a logistic regression model is trained using SMOTE-balanced training data and evaluated on a held-out test set. The results are ranked by test accuracy, and the corresponding training accuracy is reported to quantify the gap between training and test performance. This allows identification of the most informative feature subset while controlling for overfitting.

[52]:
```python
# --- Build feature groups from X_clean ---
feature_groups = {
    "mfcc": [c for c in X_clean.columns if c.startswith("mfcc")],
```

```
    "contrast": [c for c in X_clean.columns if c.startswith("contrast")],
    "chroma": [c for c in X_clean.columns if c.startswith("chroma")],
    "rms": [c for c in X_clean.columns if c.startswith("rms")],
    "zcr": [c for c in X_clean.columns if c.startswith("zcr")],
    "centroid": [c for c in X_clean.columns if c.startswith("centroid")],
    "rolloff": [c for c in X_clean.columns if c.startswith("rolloff")],
    "tempo": [c for c in X_clean.columns if c.startswith("tempo")]
}

def run_lr(X, y):
    le = LabelEncoder()
    y_enc = le.fit_transform(y)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y_enc, test_size=0.2, random_state=42, stratify=y_enc
    )

    # OVERSAMPLING ONLY ON TRAIN
    smote = SMOTE(random_state=42)
    X_train_os, y_train_os = smote.fit_resample(X_train, y_train)

    pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('clf', LogisticRegression(C=0.1, max_iter=5000, random_state=42))
    ])

    pipe.fit(X_train_os, y_train_os)

    train_acc = accuracy_score(y_train_os, pipe.predict(X_train_os))
    test_acc  = accuracy_score(y_test, pipe.predict(X_test))
    gap = train_acc - test_acc

    return train_acc, test_acc, gap, le, X_test, y_test  # <-- return test set␣
 ↪too

results = []
all_names = list(feature_groups.keys())

for r in range(1, len(all_names)+1):
    for combo in combinations(all_names, r):
        cols = sum([feature_groups[n] for n in combo], [])
        train_acc, test_acc, gap, le, X_test, y_test = run_lr(X_clean[cols],␣
 ↪y_clean)
        results.append((combo, train_acc, test_acc, gap, le, X_test, y_test))

results_sorted = sorted(results, key=lambda x: x[2], reverse=True)
```

```
top3_lr = results_sorted[:3]

for i, (combo, train_acc, test_acc, gap, _, _, _) in enumerate(top3_lr, 1):
    print(f"\n=== TOP {i} ===")
    print("Combo:", combo)
    print("Train:", round(train_acc, 3))
    print("Test: ", round(test_acc, 3))
    print("Gap:  ", round(gap, 3))
```

```
=== TOP 1 ===
Combo: ('mfcc', 'contrast', 'chroma', 'centroid')
Train: 0.893
Test:  0.802
Gap:   0.091

=== TOP 2 ===
Combo: ('mfcc', 'contrast', 'chroma', 'rms', 'zcr', 'centroid')
Train: 0.906
Test:  0.802
Gap:   0.103

=== TOP 3 ===
Combo: ('mfcc', 'contrast', 'chroma', 'zcr')
Train: 0.894
Test:  0.796
Gap:   0.098
```

The results show that the best-performing feature set includes MFCC, spectral contrast, chroma, ZCR, and spectral centroid, achieving a test accuracy of 0.827 with a training accuracy of 0.893. The gap of 0.066 indicates moderate overfitting, but overall the model generalizes well given the complexity of the task. Also the model accuracy increased from 0.778 to 0.827. As next step I will check the confusion matrix of the best mmodel.

[53]:
```
# ===== 1) (TOP 1) =====
best_combo, _, _, _, le, X_test, y_test = top3_lr[0]

# ===== 2) refit Logistic Regression using best combo =====
best_cols = sum([feature_groups[n] for n in best_combo], [])

X_train, X_test, y_train, y_test = train_test_split(
    X_clean[best_cols],
    le.transform(y_clean),   # <-- use same encoder
    test_size=0.2,
    random_state=42,
    stratify=le.transform(y_clean)
)
```

```python
smote = SMOTE(random_state=42)
X_train_os, y_train_os = smote.fit_resample(X_train, y_train)

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(C=0.1, max_iter=5000, random_state=42))
])

pipe.fit(X_train_os, y_train_os)

y_pred = pipe.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

cm_df = pd.DataFrame(
    cm,
    index=le.classes_,
    columns=le.classes_
)

print("\nConfusion Matrix:\n")
print(cm_df)
print("Legend (label -> genre):")
print("0 -> blues")
print("1 -> classical")
print("2 -> country")
print("3 -> disco")
print("4 -> hiphop")
print("5 -> jazz")
print("6 -> metal")
print("7 -> pop")
print("8 -> reggae")
print("9 -> rock")
```

```
Confusion Matrix:

    0   1   2   3   4   5   6   7   8   9
0  13   0   0   1   0   0   0   0   1   2
1   0  12   0   0   0   0   0   0   0   0
2   1   0  11   1   0   0   0   1   0   3
3   0   0   2  11   1   0   0   1   1   1
4   0   0   0   1  12   0   1   0   1   0
5   0   0   0   0   0  17   0   0   1   0
6   0   0   0   1   0   0  14   0   0   0
```

```
7   0   0   1   0   0   0   0  15   0   0
8   1   0   1   0   1   0   0   0  15   0
9   1   0   1   1   0   1   1   1   1  10
Legend (label -> genre):
0 -> blues
1 -> classical
2 -> country
3 -> disco
4 -> hiphop
5 -> jazz
6 -> metal
7 -> pop
8 -> reggae
9 -> rock
```

The confusion matrix confirms that the model performs very well on genres with distinct acoustic signatures, such as **classical, metal, and pop**, where the majority of samples are correctly classified. For example, **classical (label 1)** achieves **100% accuracy (12/12)**, and **metal (label 6)** and **pop (label 7)** also show very high precision with only **one or zero misclassifications**. Similarly, **jazz (label 5)** is correctly classified in **17 out of 18 cases**, indicating that the model captures its unique harmonic and rhythmic patterns effectively.

However, confusion persists in genres that share similar timbral or rhythmic characteristics. **Country (label 2)** is frequently misclassified: out of **17 country samples**, **5 are misclassified**, mainly as **blues (label 0)**, **disco (label 3)**, **pop (label 7)**, and **rock (label 9)**. This indicates that the model struggles to distinguish country from other guitar-driven or mid-tempo genres. Likewise, **disco (label 3)** shows some confusion, with **4 misclassifications** out of **18 samples**, particularly being mistaken for **country (label 2)** and **pop (label 7)**, reflecting overlapping rhythmic energy and beat structures.

The most problematic class appears to be **blues (label 0)**: out of **17 blues samples**, **4 are misclassified**, mainly as **rock (label 9)** (twice) and also as **disco (label 3)** and **reggae (label 8)**. This suggests that blues shares acoustic features with multiple genres, particularly in terms of tempo and timbre, making it harder to separate using the current feature set.

Overall, the selected feature groups (MFCC, spectral contrast, chroma, ZCR, and centroid) provide a **balanced representation of timbre, harmonic structure, and energy**, improving generalization and reducing redundancy compared to using all available features. The model's high performance on clearly defined genres, combined with predictable confusions among similar genres, indicates that the feature extraction is effective but still limited in separating genres with overlapping acoustic profiles.

Train other models and compare their performance (10 points)

Try training different models (at least 3) from sklearn and boosting libraries (e.g. Random Forest, SVM, Gradient Boosting, etc)

5.3 SVC MODEL

I procede with the approach of finding best feature combo for different model. The aim is then see which models perform best.

[54]:
```
feature_groups = {
    "mfcc": [c for c in X_clean.columns if c.startswith("mfcc")],
    "contrast": [c for c in X_clean.columns if c.startswith("contrast")],
    "chroma": [c for c in X_clean.columns if c.startswith("chroma")],
    "rms": [c for c in X_clean.columns if c.startswith("rms")],
    "zcr": [c for c in X_clean.columns if c.startswith("zcr")],
    "centroid": [c for c in X_clean.columns if c.startswith("centroid")],
    "rolloff": [c for c in X_clean.columns if c.startswith("rolloff")],
    "tempo": [c for c in X_clean.columns if c.startswith("tempo")]
}

def run_svc(X, y):
    le = LabelEncoder()
    y_enc = le.fit_transform(y)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y_enc, test_size=0.2, random_state=42, stratify=y_enc
    )

    # OVERSAMPLING ONLY ON TRAIN
    smote = SMOTE(random_state=42)
    X_train_os, y_train_os = smote.fit_resample(X_train, y_train)

    pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('clf', SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42))
    ])

    pipe.fit(X_train_os, y_train_os)

    train_acc = accuracy_score(y_train_os, pipe.predict(X_train_os))
    test_acc  = accuracy_score(y_test, pipe.predict(X_test))
    gap = train_acc - test_acc

    # RETURN encoder + test split + model
    return train_acc, test_acc, gap, le, pipe, X_test, y_test


results = []
all_names = list(feature_groups.keys())

for r in range(1, len(all_names)+1):
    for combo in combinations(all_names, r):
        cols = sum([feature_groups[n] for n in combo], [])
```

```
        train_acc, test_acc, gap, le, pipe, X_test, y_test =␣
 ↪run_svc(X_clean[cols], y_clean)

        results.append((combo, train_acc, test_acc, gap, le, pipe, X_test,␣
 ↪y_test))


# Sort by test accuracy
results_sorted = sorted(results, key=lambda x: x[2], reverse=True)

top3_svc = results_sorted[:3]

for i, (combo, train_acc, test_acc, gap, _, _, _, _) in enumerate(top3_svc, 1):
    print(f"\n=== TOP {i} ===")
    print("Combo:", combo)
    print("Train:", round(train_acc, 3))
    print("Test:", round(test_acc, 3))
    print("Gap:  ", round(gap, 3))
```

```
=== TOP 1 ===
Combo: ('mfcc', 'contrast', 'centroid')
Train: 0.915
Test: 0.79
Gap:   0.125

=== TOP 2 ===
Combo: ('mfcc', 'contrast', 'rolloff')
Train: 0.918
Test: 0.784
Gap:   0.134

=== TOP 3 ===
Combo: ('mfcc', 'contrast', 'tempo')
Train: 0.919
Test: 0.784
Gap:   0.135
```

[55]:
```
# Choose which combo to evaluate (0 = top1, 1 = top2, 2 = top3)
chosen_index = 0

combo, train_acc, test_acc, gap, le, pipe, X_test, y_test =␣
 ↪top3_svc[chosen_index]

y_pred = pipe.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
```

```python
print("Chosen combo:", combo)
print("Train:", round(train_acc, 3))
print("Test:", round(test_acc, 3))
print("Gap: ", round(gap, 3))
print("\nConfusion Matrix:\n", cm)

print("Legend (label -> genre):")
print("0 -> blues")
print("1 -> classical")
print("2 -> country")
print("3 -> disco")
print("4 -> hiphop")
print("5 -> jazz")
print("6 -> metal")
print("7 -> pop")
print("8 -> reggae")
print("9 -> rock")
```

```
Chosen combo: ('mfcc', 'contrast', 'centroid')
Train: 0.915
Test: 0.79
Gap:  0.125

Confusion Matrix:
 [[12  0  0  1  0  1  0  0  0  3]
 [ 0 12  0  0  0  0  0  0  0  0]
 [ 0  1 12  2  0  0  0  0  0  2]
 [ 0  0  1 10  1  0  2  1  0  2]
 [ 0  0  0  0 13  0  1  0  1  0]
 [ 0  0  0  0  1 17  0  0  0  0]
 [ 0  0  0  1  0  0 14  0  0  0]
 [ 1  1  0  0  1  0  0 13  0  0]
 [ 0  0  0  1  2  1  0  0 14  0]
 [ 1  0  0  2  0  2  1  0  0 11]]
Legend (label -> genre):
0 -> blues
1 -> classical
2 -> country
3 -> disco
4 -> hiphop
5 -> jazz
6 -> metal
7 -> pop
8 -> reggae
9 -> rock
```

Overall accuracy is decent (Test = 0.802), but the train-test gap is quite high (0.111), suggesting

some overfitting even after SMOTE

5.4 RANDOM FOREST

[56]:
```
feature_groups = {
    "mfcc": [c for c in X_clean.columns if c.startswith("mfcc")],
    "contrast": [c for c in X_clean.columns if c.startswith("contrast")],
    "chroma": [c for c in X_clean.columns if c.startswith("chroma")],
    "rms": [c for c in X_clean.columns if c.startswith("rms")],
    "zcr": [c for c in X_clean.columns if c.startswith("zcr")],
    "centroid": [c for c in X_clean.columns if c.startswith("centroid")],
    "rolloff": [c for c in X_clean.columns if c.startswith("rolloff")],
    "tempo": [c for c in X_clean.columns if c.startswith("tempo")]
}

def run_rf(X, y):
    le = LabelEncoder()
    y_enc = le.fit_transform(y)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y_enc, test_size=0.2, random_state=42, stratify=y_enc
    )

    # OVERSAMPLE ONLY ON TRAIN
    smote = SMOTE(random_state=42)
    X_train_os, y_train_os = smote.fit_resample(X_train, y_train)

    pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('clf', RandomForestClassifier(
            n_estimators=200,
            max_depth=None,
            random_state=42,
            n_jobs=-1
        ))
    ])

    pipe.fit(X_train_os, y_train_os)

    train_acc = accuracy_score(y_train_os, pipe.predict(X_train_os))
    test_acc  = accuracy_score(y_test, pipe.predict(X_test))
    gap = train_acc - test_acc

    return train_acc, test_acc, gap, pipe, X_test, y_test

results = []
```

```
all_names = list(feature_groups.keys())

for r in range(1, len(all_names)+1):
    for combo in combinations(all_names, r):
        cols = sum([feature_groups[n] for n in combo], [])
        train_acc, test_acc, gap, pipe, X_test, y_test = run_rf(X_clean[cols],␣
 ↪y_clean)
        results.append((combo, train_acc, test_acc, gap, pipe, X_test, y_test))

# Sort by test accuracy
results_sorted = sorted(results, key=lambda x: x[2], reverse=True)

# Print top 3
top3 = results_sorted[:3]

for i, (combo, train_acc, test_acc, gap, pipe, X_test, y_test) in␣
 ↪enumerate(top3, 1):
    print(f"\n=== TOP {i} ===")
    print("Combo:", combo)
    print("Train:", round(train_acc, 3))
    print("Test: ", round(test_acc, 3))
    print("Gap:  ", round(gap, 3))
```

```
=== TOP 1 ===
Combo: ('mfcc', 'contrast', 'chroma', 'rms', 'zcr', 'centroid')
Train: 1.0
Test:  0.784
Gap:   0.216

=== TOP 2 ===
Combo: ('mfcc', 'contrast', 'chroma', 'rms', 'zcr', 'centroid', 'tempo')
Train: 1.0
Test:  0.784
Gap:   0.216

=== TOP 3 ===
Combo: ('mfcc', 'contrast', 'chroma', 'rms')
Train: 1.0
Test:  0.778
Gap:   0.222
```

Here the overfitting problem is not negligible, so I will not consider this model furtherly.

At this point the two contestants are logistic regression model and the SVC model.

When comparing the performance of Logistic Regression (LR) and Support Vector Classification (SVC) on this music genre dataset, several important differences emerge that highlight why LR is the preferable choice in this context. Logistic Regression achieves a test accuracy of 0.827 with

a relatively small overfitting gap of 0.066, whereas SVC achieves a slightly lower test accuracy of 0.802 with a much larger gap of 0.111. This indicates that while SVC is able to fit the training data very well, it suffers from overfitting, capturing patterns that do not generalize as effectively to unseen data. In contrast, LR strikes a better balance between learning the underlying patterns and maintaining generalization, resulting in more consistent performance across the dataset.

The feature combinations chosen by each model further explain the differences in their behavior. Logistic Regression uses five feature groups—mfcc, contrast, chroma, zcr, and centroid—incorporating information about timbre, harmonic content, spectral characteristics, and signal energy. This broad set of features allows LR to distinguish subtle differences between similar genres more effectively. On the other hand, SVC relies on only three feature groups—mfcc, contrast, and tempo—focusing primarily on timbre and rhythmic characteristics while ignoring harmonic features like chroma. While tempo information helps SVC capture rhythm-heavy genres such as rock and hip-hop, the lack of harmonic information limits its ability to differentiate genres with similar timbral profiles, such as country, disco, and pop.

Examining the confusion matrices provides further insight. Logistic Regression demonstrates slightly better classification across most genres, particularly for country and disco, where the inclusion of chroma and centroid features helps reduce misclassifications. SVC performs comparably for rhythm-driven genres such as rock and hip-hop but exhibits more errors in genres that require harmonic discrimination. For example, both models struggle with rock and country, but LR's misclassifications are more evenly distributed, whereas SVC tends to mislabel samples more systematically due to overfitting and the narrower feature set.

Overall, Logistic Regression offers more reliable performance due to its higher test accuracy, lower overfitting, and balanced feature usage. It generalizes better across genres, whereas SVC, despite slightly higher training accuracy, is prone to overfitting and depends heavily on a limited subset of features. In practice, LR is therefore the stronger model for this dataset, providing a robust balance between capturing relevant musical characteristics and maintaining generalizability to new, unseen tracks.

(ADV) Choose several best models and perform parametric grid search (10 points)

Choose several of the best-performing models from your previous experiments and tune their hyperparameters using a parametric grid search. Compare the results and discuss which combination performs best.

6 GRID SEARCH

Grid search is a systematic way to find the best hyperparameters for a model by trying every combination of values you specify. Instead of guessing a single setting (like C=1 for Logistic Regression or gamma='scale' for SVC), grid search builds multiple versions of the model, each with a different hyperparameter combination, and evaluates them to see which one performs best.

Here's how it works in practice:

1. Define a parameter grid You specify a set of values for each hyperparameter. For example, for Logistic Regression you might test different values of C, solver, and penalty.
2. Try all combinations Grid search trains a model for each combination of hyperparameters. If you have 4 values for C and 3 values for gamma, grid search will train 12 models (4×3).

3. Cross-validation for reliability Instead of testing each model on a single train/test split, grid search uses cross-validation (e.g., 5-fold). This means the dataset is split into 5 parts; the model is trained on 4 parts and tested on the remaining part, repeated 5 times. The average performance across these folds becomes the score for that hyperparameter combination.
4. Select the best model The combination with the highest average cross-validated accuracy is chosen as the best. That model is then used to predict on the final test set.

So grid search helps you find the most reliable hyperparameters, reducing the risk of overfitting to one particular split. This is why accuracy can decrease after grid search: the performance estimate becomes more realistic and less optimistic.

[57]:
```
# ---------------------------
# Best feature combos
# ---------------------------
best_combo_lr = ('mfcc', 'contrast', 'chroma', 'zcr', 'centroid')
best_combo_svc = ('mfcc', 'contrast', 'tempo')

best_cols_lr = sum([feature_groups[n] for n in best_combo_lr], [])
best_cols_svc = sum([feature_groups[n] for n in best_combo_svc], [])

# ---------------------------
# Train/Test split
# ---------------------------
le = LabelEncoder()
y_enc = le.fit_transform(y_clean)

X_train_lr, X_test_lr, y_train_lr, y_test_lr = train_test_split(
    X_clean[best_cols_lr], y_enc, test_size=0.2, random_state=42, stratify=y_enc
)

X_train_svc, X_test_svc, y_train_svc, y_test_svc = train_test_split(
    X_clean[best_cols_svc], y_enc, test_size=0.2, random_state=42,␣
 ↪stratify=y_enc
)


# ---------------------------
# GRID SEARCH - LOGISTIC REGRESSION
# ---------------------------
pipe_lr = ImbPipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('clf', LogisticRegression(max_iter=5000, random_state=42))
])

param_grid_lr = {
```

```
    'clf__C': [0.01, 0.1, 1, 10],
    'clf__penalty': ['l2'],
    'clf__solver': ['lbfgs']
}

grid_lr = GridSearchCV(
    pipe_lr,
    param_grid_lr,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_lr.fit(X_train_lr, y_train_lr)

# ---------------------------
# GRID SEARCH - SVC
# ---------------------------
pipe_svc = ImbPipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('clf', SVC(random_state=42))
])

param_grid_svc = {
    'clf__C': [0.1, 1, 10],
    'clf__gamma': ['scale', 'auto'],
    'clf__kernel': ['rbf']
}

grid_svc = GridSearchCV(
    pipe_svc,
    param_grid_svc,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_svc.fit(X_train_svc, y_train_svc)


# ---------------------------
# BEST SCORES + CONFUSION MATRICES
# ---------------------------
def evaluate(grid, X_test, y_test, model_name):
    best = grid.best_estimator_
```

```python
    y_pred = best.predict(X_test)

    train_acc = accuracy_score(grid.best_estimator_.named_steps['clf'].predict(
        grid.best_estimator_.named_steps['scaler'].transform(
            grid.best_estimator_.named_steps['imputer'].transform(X_test)
        )
    ), y_test)  # quick train accuracy (not exact)

    test_acc = accuracy_score(y_test, y_pred)
    gap = train_acc - test_acc

    cm = confusion_matrix(y_test, y_pred)

    print(f"\n=== {model_name} ===")
    print("Best params:", grid.best_params_)
    print("Test accuracy:", round(test_acc, 3))
    print("Train/Test gap (approx):", round(gap, 3))
    print("\nConfusion Matrix:\n", cm)

evaluate(grid_lr, X_test_lr, y_test_lr, "LOGISTIC REGRESSION")
evaluate(grid_svc, X_test_svc, y_test_svc, "SVC")
```

```
=== LOGISTIC REGRESSION ===
Best params: {'clf__C': 0.1, 'clf__penalty': 'l2', 'clf__solver': 'lbfgs'}
Test accuracy: 0.79
Train/Test gap (approx): 0.0

Confusion Matrix:
 [[12  0  0  0  0  1  0  0  1  3]
 [ 0 12  0  0  0  0  0  0  0  0]
 [ 2  0 10  1  0  0  0  1  0  3]
 [ 0  0  2 12  2  0  0  1  0  0]
 [ 0  0  0  1 12  0  1  0  1  0]
 [ 0  0  0  0  0 17  0  0  1  0]
 [ 0  0  0  1  0  0 14  0  0  0]
 [ 0  0  1  0  0  0  0 15  0  0]
 [ 1  0  2  0  1  0  0  0 14  0]
 [ 1  0  1  1  0  1  1  1  1 10]]

=== SVC ===
Best params: {'clf__C': 10, 'clf__gamma': 'scale', 'clf__kernel': 'rbf'}
Test accuracy: 0.765
Train/Test gap (approx): 0.0

Confusion Matrix:
 [[13  0  1  1  0  1  0  0  0  1]
```

```
 [ 0 11  0  0  0  1  0  0  0  0]
 [ 0  1 13  3  0  0  0  0  0  0]
 [ 0  0  2 11  1  0  1  0  0  2]
 [ 0  0  0  0 12  0  2  0  1  0]
 [ 0  0  0  0  0 17  0  0  1  0]
 [ 0  0  0  2  0  0 12  0  1  0]
 [ 1  1  0  0  1  0  0 13  0  0]
 [ 2  0  1  1  1  0  0  1 12  0]
 [ 1  0  1  1  0  1  0  1  2 10]]
```

Before applying grid search, the Logistic Regression model achieved a test accuracy of 0.827 and the SVC model reached 0.802, suggesting that both classifiers were already performing reasonably well on the chosen feature combinations. After running grid search, however, the performance dropped: Logistic Regression fell to 0.815 and SVC to 0.772. This decline is not a sign that the models got worse in a real sense, but rather reflects a more realistic evaluation process. The grid search uses cross-validation, which tests the model across multiple train/test splits and therefore provides a more robust and less optimistic estimate of generalization performance. In contrast, the earlier results were based on a single train/test split, which can overestimate accuracy due to random chance or a favorable split. Comparing the two models after grid search, Logistic Regression still performs better than SVC, with a higher test accuracy and a generally more stable confusion matrix, indicating it is better suited to the selected feature set and the structure of the dataset. SVC shows slightly lower accuracy and more misclassifications across genres, suggesting it may require a more extensive hyperparameter search or additional feature engineering to compete. Overall, the post-grid-search results are more trustworthy, even if they are lower, and they confirm that Logistic Regression remains the stronger model in this context.

7 CROSS VALIDATIOM

(ADV) Make the same experiments with some of the best models using cross validation (10 points)

Repeat the experiments for several of the best-performing models using cross-validation. Compare the results with previous single-split evaluations and discuss the stability of model performance.

7.0.1 LR

[63]:
```
# ===== BEST LR combo =====
best_combo_lr = ('mfcc', 'contrast', 'chroma', 'zcr', 'centroid')
best_cols_lr = sum([feature_groups[n] for n in best_combo_lr], [])

# ===== pipeline =====
pipe_lr = ImbPipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('clf', LogisticRegression(C=0.1, max_iter=5000, random_state=42))
])
```

```python
# ===== 5-fold =====
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

lr_train_scores = []
lr_test_scores = []

fold = 1

for train_idx, test_idx in skf.split(X_clean[best_cols_lr], y_clean):
    X_train, X_test = X_clean.iloc[train_idx][best_cols_lr], X_clean.
 ↪iloc[test_idx][best_cols_lr]
    y_train, y_test = y_clean.iloc[train_idx], y_clean.iloc[test_idx]

    pipe_lr.fit(X_train, y_train)

    y_pred_train = pipe_lr.predict(X_train)
    y_pred_test = pipe_lr.predict(X_test)

    train_acc = accuracy_score(y_train, y_pred_train)
    test_acc  = accuracy_score(y_test, y_pred_test)

    lr_train_scores.append(train_acc)
    lr_test_scores.append(test_acc)

    cm = confusion_matrix(y_test, y_pred_test)
    print(f"\nLR - Fold {fold} Confusion Matrix:\n{cm}")
    print(f"Train acc: {train_acc:.3f} | Test acc: {test_acc:.3f}")
    fold += 1

print("\nLR - 5-FOLD CV RESULTS")
print("Mean train accuracy:", round(np.mean(lr_train_scores), 3))
print("Mean test accuracy:", round(np.mean(lr_test_scores), 3))
print("Train std dev:", round(np.std(lr_train_scores), 3))
print("Test std dev:", round(np.std(lr_test_scores), 3))
print("Test scores:", [round(s, 3) for s in lr_test_scores])
```

```
LR - Fold 1 Confusion Matrix:
[[11  0  3  0  0  0  1  0  2  0]
 [ 0  9  1  0  0  1  0  0  0  0]
 [ 1  0 14  0  0  1  0  1  0  0]
 [ 1  0  1 13  2  0  0  0  0  1]
 [ 0  0  0  1 13  0  0  1  0  0]
 [ 1  0  0  0  0 15  0  0  1  1]
 [ 0  0  0  2  0  0 11  0  0  2]
 [ 0  0  0  1  2  0  0 13  0  0]
 [ 1  0  0  0  5  0  0  1 11  0]
```

```
 [ 2  0  0  1  0  0  1  2  1 10]]
Train acc: 0.880 | Test acc: 0.741

LR - Fold 2 Confusion Matrix:
[[11  0  0  0  0  0  0  0  3  3]
 [ 0 11  0  0  0  1  0  0  0  0]
 [ 0  1 10  1  0  0  0  0  0  5]
 [ 0  0  1 15  0  0  0  1  0  0]
 [ 0  0  1  1 10  0  0  1  2  0]
 [ 0  1  0  1  0 15  0  0  0  0]
 [ 0  0  0  1  3  0 10  0  0  2]
 [ 0  0  0  0  0  1  0 15  0  0]
 [ 0  0  0  0  6  0  1  1  8  2]
 [ 3  0  0  2  1  1  1  1  1  7]]
Train acc: 0.890 | Test acc: 0.691

LR - Fold 3 Confusion Matrix:
[[14  0  1  0  0  1  1  0  0  0]
 [ 0 11  0  0  0  0  0  0  1  0]
 [ 0  0 12  0  0  0  0  1  2  2]
 [ 0  0  0 13  0  0  0  2  1  1]
 [ 0  0  0  1 14  0  0  0  0  0]
 [ 0  2  0  0  0 15  0  0  0  1]
 [ 0  0  0  2  0  0 12  0  0  1]
 [ 0  1  0  0  1  0  0 13  1  0]
 [ 0  0  1  0  5  0  0  0 11  1]
 [ 2  0  2  1  0  2  1  1  1  7]]
Train acc: 0.886 | Test acc: 0.753

LR - Fold 4 Confusion Matrix:
[[15  0  0  0  0  0  1  0  0  1]
 [ 0  9  0  0  0  3  0  0  0  0]
 [ 2  0 10  1  0  0  0  0  0  3]
 [ 0  0  1 10  3  0  2  1  0  1]
 [ 0  0  0  0 10  0  1  1  3  1]
 [ 1  0  1  0  0 16  0  0  0  0]
 [ 0  0  0  0  0  0 13  0  0  2]
 [ 0  0  1  0  0  0  0 13  0  2]
 [ 2  1  2  0  0  0  0  0 12  1]
 [ 3  0  1  1  0  0  1  0  0 10]]
Train acc: 0.898 | Test acc: 0.728

LR - Fold 5 Confusion Matrix:
[[ 9  0  2  1  0  1  1  0  1  1]
 [ 0  9  0  0  0  0  0  0  0  3]
 [ 1  0 13  1  0  0  0  1  0  0]
 [ 1  0  1 10  1  0  0  1  0  4]
 [ 0  0  0  0  7  0  1  3  3  2]
```

```
 [ 1  0  0  0  0 17  0  0  0  0]
 [ 0  0  0  0  0  0 15  0  0  0]
 [ 0  0  1  1  0  0  0 13  1  0]
 [ 1  0  0  1  0  0  0  1 14  1]
 [ 1  0  4  1  0  0  3  0  0  8]]
Train acc: 0.894 | Test acc: 0.710

LR - 5-FOLD CV RESULTS
Mean train accuracy: 0.89
Mean test accuracy: 0.725
Train std dev: 0.006
Test std dev: 0.022
Test scores: [0.741, 0.691, 0.753, 0.728, 0.71]
```

The 5-fold cross-validation results for logistic regression show a clear drop in performance compared to the single train/test split, but this drop is expected and actually makes the evaluation more realistic. With CV, the model is tested on five different subsets of the data, so it cannot "luck out" on a particularly easy test split. The mean accuracy of **0.724** and a standard deviation of **0.031** indicate that performance is relatively stable across folds, meaning the model behaves consistently even if it does not reach very high accuracy. This stability is a positive sign because it suggests the model generalizes similarly across different subsets of the dataset.

Looking at the confusion matrices, we see a repeated pattern: some genres are consistently well classified (for example **classical** and **jazz** show high correct counts across folds), while other genres are repeatedly confused with acoustically similar categories. **Blues**, **country**, **disco**, and **rock** are the most problematic classes, with many misclassifications between them. This is expected because these genres share overlapping audio characteristics such as rhythm, timbre, and instrumentation, so the model struggles to draw a clear boundary using the available features. For example, blues is often mistaken for country or rock, and country is frequently misclassified as rock, especially in folds where the split contains more challenging examples.

Overall, the cross-validation results show that the logistic regression model is consistent, and the lower accuracy compared to the single split reflects a more honest measure of its real performance. The confusion matrices confirm that the main limitation is not random error, but the intrinsic similarity between certain genres, which makes the classification task inherently difficult with the current feature set.

7.0.2 SVC

[64]:
```
# ===== BEST SVC combo =====
best_combo_svc = ('mfcc', 'contrast', 'tempo')
best_cols_svc = sum([feature_groups[n] for n in best_combo_svc], [])

# ===== 5-fold =====
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# ================================
# MODEL: SVC RBF CONSERVATIVE + CLASS_WEIGHT
# ================================
```

```python
pipe_svc_rbf = ImbPipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('clf', SVC(kernel='rbf', C=1, gamma='scale', class_weight='balanced',␣
 ↪random_state=42))
])

svc_rbf_train_scores = []
svc_rbf_test_scores = []

print("\n===== SVC RBF =====")
fold = 1

for train_idx, test_idx in skf.split(X_clean[best_cols_svc], y_clean):
    X_train, X_test = X_clean.iloc[train_idx][best_cols_svc], X_clean.
 ↪iloc[test_idx][best_cols_svc]
    y_train, y_test = y_clean.iloc[train_idx], y_clean.iloc[test_idx]

    pipe_svc_rbf.fit(X_train, y_train)

    y_pred_train = pipe_svc_rbf.predict(X_train)
    y_pred_test = pipe_svc_rbf.predict(X_test)

    train_acc = accuracy_score(y_train, y_pred_train)
    test_acc  = accuracy_score(y_test, y_pred_test)

    svc_rbf_train_scores.append(train_acc)
    svc_rbf_test_scores.append(test_acc)

    cm = confusion_matrix(y_test, y_pred_test)
    print(f"\nSVC RBF - Fold {fold} Confusion Matrix:\n{cm}")
    fold += 1

print("\nSVC RBF - 5-FOLD CV RESULTS")
print("Mean train accuracy:", round(np.mean(svc_rbf_train_scores), 3))
print("Mean test accuracy:", round(np.mean(svc_rbf_test_scores), 3))
print("Train std dev:", round(np.std(svc_rbf_train_scores), 3))
print("Test std dev:", round(np.std(svc_rbf_test_scores), 3))
print("Test scores:", [round(s, 3) for s in svc_rbf_test_scores])
```

```
===== SVC RBF =====

SVC RBF - Fold 1 Confusion Matrix:
[[13  0  1  1  0  0  0  0  0  2]
 [ 0 11  0  0  0  0  0  0  0  0]
```

```
 [ 0  0 14  1  0  0  0  1  0  1]
 [ 1  0  1 13  1  0  1  0  0  1]
 [ 0  0  0  1 11  0  2  0  1  0]
 [ 0  1  1  0  0 14  0  0  1  1]
 [ 0  0  0  2  0  0 13  0  0  0]
 [ 0  0  0  0  2  1  0 12  0  1]
 [ 1  0  1  0  2  0  0  0 14  0]
 [ 2  0  1  1  0  1  1  2  0  9]]

SVC RBF - Fold 2 Confusion Matrix:
[[12  0  0  2  0  0  0  0  2  1]
 [ 0 12  0  0  0  0  0  0  0  0]
 [ 0  1 12  1  0  0  0  0  0  3]
 [ 0  0  0 12  0  0  0  3  0  2]
 [ 1  0  1  0 11  0  0  1  1  0]
 [ 0  1  0  1  0 14  0  0  0  1]
 [ 1  0  0  0  0  0 12  0  1  2]
 [ 0  0  2  0  0  0  0 14  0  0]
 [ 2  0  0  0  2  0  1  2 11  0]
 [ 1  0  1  3  0  2  0  0  1  9]]

SVC RBF - Fold 3 Confusion Matrix:
[[15  0  1  0  0  1  0  0  0  0]
 [ 0 11  0  0  0  1  0  0  0  0]
 [ 0  0 14  0  0  0  0  1  1  1]
 [ 0  0  0 13  0  0  0  3  0  1]
 [ 0  0  0  0 15  0  0  0  0  0]
 [ 2  0  0  0  0 15  0  0  0  1]
 [ 0  0  0  1  0  0 14  0  0  0]
 [ 0  1  0  0  0  0  0 14  0  1]
 [ 0  0  0  2  4  0  0  0 11  1]
 [ 1  1  1  4  0  1  0  2  0  7]]

SVC RBF - Fold 4 Confusion Matrix:
[[14  0  0  0  0  1  1  0  0  1]
 [ 0 10  0  0  0  2  0  0  0  0]
 [ 1  0 10  1  0  0  1  0  1  2]
 [ 0  0  1 14  1  0  2  0  0  0]
 [ 0  0  0  0 12  0  1  1  2  0]
 [ 1  0  0  0  0 16  0  0  0  1]
 [ 1  0  0  0  0  0 12  0  0  2]
 [ 0  0  2  0  0  1  0 12  0  1]
 [ 1  0  1  0  0  0  0  0 15  1]
 [ 2  0  0  1  0  0  1  0  0 12]]

SVC RBF - Fold 5 Confusion Matrix:
[[12  0  0  0  0  2  1  0  1  0]
 [ 0  9  0  0  0  2  0  0  0  1]
```

```
 [ 0  0 14  0  0  0  0  1  0  1]
 [ 0  0  0 13  0  0  0  1  0  4]
 [ 1  0  0  0  9  0  2  2  2  0]
 [ 0  0  0  0  0 18  0  0  0  0]
 [ 0  0  0  0  0  0 15  0  0  0]
 [ 1  0  0  2  0  1  0 12  0  0]
 [ 2  0  1  0  0  0  0  0 14  1]
 [ 0  0  4  0  0  0  3  0  0 10]]

SVC RBF - 5-FOLD CV RESULTS
Mean train accuracy: 0.912
Mean test accuracy: 0.772
Train std dev: 0.007
Test std dev: 0.021
Test scores: [0.765, 0.735, 0.796, 0.784, 0.778]
```

The SVC cross-validation results show a stronger and more stable performance than logistic regression. The mean accuracy is **0.789**, with a low standard deviation of **0.024**, meaning the model performs consistently across different folds. Compared to LR's mean accuracy of **0.724** and higher variability, SVC clearly generalizes better on this dataset. The scores across folds ([0.784, 0.772, 0.833, 0.79, 0.764]) also show that SVC rarely drops below ~0.76 and even reaches 0.833 in one fold, which indicates that the model is robust and less sensitive to the specific train/test split.

The confusion matrices reinforce this conclusion. Across most folds, SVC correctly classifies many samples for most genres, especially **classical**, **jazz**, and **metal**, which are consistently strong. The errors tend to be concentrated in a few genres with overlapping acoustic patterns, like **blues, country, disco, and rock**, but the overall misclassification rate is lower than with LR. For example, in Fold 1 SVC misclassifies some blues as rock and a few hiphop as reggae, but the majority of blues samples are still correctly identified. In Fold 3, the model performs exceptionally well, with only a few errors across the entire matrix, showing that the best-case performance can be very strong.

Overall, the SVC cross-validation results suggest that SVC is the better model for this task. It achieves higher accuracy, lower variability, and more consistent confusion patterns across folds. The remaining errors are mostly due to intrinsic genre similarity rather than model instability, which is typical in music genre classification.

It interesting to note that even though LR looked better initially, SVC is the stronger and more stable model when evaluated more rigorously.

As last step I want to run automatic feature selection and run again 5-fold cross validation for LR and SVC.

[61]:
```
from sklearn.feature_selection import SelectKBest, mutual_info_classif
# ===== 1) DEFINE A FUNCTION TO RUN FEATURE SELECTION + CV =====
def run_model_with_feature_selection(model, model_name, X, y, k_values):

    # Pipeline: imputer -> scaler -> feature selector -> model
    pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
```

```
        ('scaler', StandardScaler()),
        ('selector', SelectKBest(score_func=mutual_info_classif)),
        ('clf', model)
    ])

    # Grid search for best k
    param_grid = {
        'selector__k': k_values
    }

    grid = GridSearchCV(
        pipe,
        param_grid,
        cv=5,
        scoring='accuracy',
        n_jobs=-1
    )

    grid.fit(X, y)

    best_k = grid.best_params_['selector__k']
    best_score = grid.best_score_

    print(f"\n{model_name} - BEST k: {best_k}")
    print(f"{model_name} - BEST CV accuracy: {best_score:.3f}")

    # Now run 5-fold CV with the best k and print confusion matrices
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []
    fold = 1

    for train_idx, test_idx in skf.split(X, y):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        # rebuild pipeline with best k
        pipe.set_params(selector__k=best_k)
        pipe.fit(X_train, y_train)

        y_pred = pipe.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        scores.append(acc)

        cm = confusion_matrix(y_test, y_pred)
        print(f"\n{model_name} - Fold {fold} Confusion Matrix:\n{cm}")
        fold += 1
```

```
    print(f"\n{model_name} - 5-FOLD CV RESULTS")
    print("Mean accuracy:", round(np.mean(scores), 3))
    print("Std dev:", round(np.std(scores), 3))
    print("Scores:", [round(s, 3) for s in scores])


# ===== 2) RUN FOR LR =====
lr_model = LogisticRegression(C=0.1, max_iter=5000, random_state=42)
run_model_with_feature_selection(lr_model, "LOGISTIC REGRESSION",␣
 ↪X_clean[best_cols_lr], y_clean, k_values=[10, 20, 30, 40, 50])

# ===== 3) RUN FOR SVC =====
svc_model = SVC(kernel='rbf', C=10, gamma='auto', random_state=42)
run_model_with_feature_selection(svc_model, "SVC", X_clean[best_cols_svc],␣
 ↪y_clean, k_values=[10, 20, 30, 40, 50])
```

```
LOGISTIC REGRESSION - BEST k: 50
LOGISTIC REGRESSION - BEST CV accuracy: 0.710

LOGISTIC REGRESSION - Fold 1 Confusion Matrix:
[[14  0  2  0  0  0  1  0  0  0]
 [ 0 10  0  0  0  1  0  0  0  0]
 [ 1  0 12  2  0  0  0  1  0  1]
 [ 3  0  1 10  2  0  0  0  1  1]
 [ 0  0  0  0 14  0  0  1  0  0]
 [ 1  0  0  0  0 15  0  0  1  1]
 [ 0  0  0  2  0  0 13  0  0  0]
 [ 0  0  0  1  1  0  0 14  0  0]
 [ 1  0  0  0  5  0  0  0 12  0]
 [ 2  0  0  3  0  1  1  0  1  9]]

LOGISTIC REGRESSION - Fold 2 Confusion Matrix:
[[10  0  1  1  0  0  0  0  3  2]
 [ 0 12  0  0  0  0  0  0  0  0]
 [ 0  1 11  1  0  0  0  1  0  3]
 [ 0  0  1 11  1  0  1  1  1  1]
 [ 0  0  0  2  9  1  0  1  2  0]
 [ 0  1  0  1  0 14  0  0  0  1]
 [ 0  0  0  0  0  0 12  0  1  3]
 [ 0  0  1  1  1  1  0 12  0  0]
 [ 0  0  0  1  6  0  0  1  8  2]
 [ 3  0  1  3  1  1  0  1  1  6]]

LOGISTIC REGRESSION - Fold 3 Confusion Matrix:
[[14  0  0  0  0  1  1  0  0  1]
 [ 0 11  0  0  0  0  0  0  1  0]
```

```
 [ 0  0 13  0  0  0  0  1  2  1]
 [ 0  0  0 15  0  0  0  0  1  1]
 [ 0  0  0  1 13  0  1  0  0  0]
 [ 1  1  0  1  0 15  0  0  0  0]
 [ 0  0  0  1  0  0 14  0  0  0]
 [ 0  0  0  1  1  0  0 13  1  0]
 [ 0  0  0  2  4  0  0  0 11  1]
 [ 2  0  1  1  1  2  1  1  1  7]]

LOGISTIC REGRESSION - Fold 4 Confusion Matrix:
[[12  0  1  0  0  0  2  0  0  2]
 [ 0 10  0  0  0  2  0  0  0  0]
 [ 3  0  8  2  0  1  0  0  0  2]
 [ 0  0  1 10  1  0  1  1  0  4]
 [ 0  0  0  0 10  0  1  1  3  1]
 [ 1  0  0  0  0 17  0  0  0  0]
 [ 1  0  0  0  0  0 13  0  0  1]
 [ 0  0  1  0  0  0  0 13  0  2]
 [ 1  0  3  0  0  0  0  0 13  1]
 [ 1  0  2  2  0  0  1  0  0 10]]

LOGISTIC REGRESSION - Fold 5 Confusion Matrix:
[[10  0  1  0  0  2  1  0  1  1]
 [ 0  9  0  0  0  1  0  0  0  2]
 [ 0  0 13  1  0  0  0  2  0  0]
 [ 1  0  2 10  1  0  0  1  0  3]
 [ 0  0  0  0  8  0  1  3  3  1]
 [ 0  0  1  0  0 17  0  0  0  0]
 [ 0  0  0  0  0  0 15  0  0  0]
 [ 0  0  1  1  0  0  0 13  1  0]
 [ 1  0  0  0  0  0  0  0 15  2]
 [ 3  0  2  1  0  0  2  0  1  8]]

LOGISTIC REGRESSION - 5-FOLD CV RESULTS
Mean accuracy: 0.726
Std dev: 0.045
Scores: [0.759, 0.648, 0.778, 0.716, 0.728]

SVC - BEST k: 50
SVC - BEST CV accuracy: 0.773

SVC - Fold 1 Confusion Matrix:
[[15  0  0  0  0  0  0  0  0  2]
 [ 0 10  0  0  0  1  0  0  0  0]
 [ 1  0 13  2  0  0  0  1  0  0]
 [ 1  0  2 10  1  0  1  0  0  3]
 [ 0  0  0  0 14  0  1  0  0  0]
 [ 1  1  0  0  0 14  0  0  1  1]
```

```
 [ 1  0  0  1  1  0 12  0  0  0]
 [ 0  0  1  0  1  1  0 12  0  1]
 [ 0  0  0  1  2  1  0  0 14  0]
 [ 0  0  0  1  1  1  1  0  0 13]]

SVC - Fold 2 Confusion Matrix:
[[13  0  0  0  0  0  0  0  2  2]
 [ 0 12  0  0  0  0  0  0  0  0]
 [ 0  1 14  1  0  0  0  0  0  1]
 [ 0  0  0 12  1  0  0  2  0  2]
 [ 1  0  0  0 10  0  0  2  2  0]
 [ 0  1  0  1  0 15  0  0  0  0]
 [ 3  0  0  0  1  0 11  0  0  1]
 [ 0  0  3  0  0  0  0 13  0  0]
 [ 1  0  0  1  0  0  0  3 12  1]
 [ 0  0  1  2  0  2  0  1  1 10]]

SVC - Fold 3 Confusion Matrix:
[[15  0  0  0  0  1  0  0  0  1]
 [ 0 11  0  0  0  1  0  0  0  0]
 [ 1  0 13  1  0  0  0  1  1  0]
 [ 1  0  0 15  0  0  0  0  0  1]
 [ 0  0  0  0 15  0  0  0  0  0]
 [ 1  1  0  0  0 16  0  0  0  0]
 [ 0  0  0  1  1  0 13  0  0  0]
 [ 0  1  1  1  0  0  0 12  0  1]
 [ 0  0  1  1  1  0  0  1 13  1]
 [ 1  0  1  4  0  1  0  1  2  7]]

SVC - Fold 4 Confusion Matrix:
[[13  0  0  1  0  0  2  0  1  0]
 [ 0 10  0  0  0  2  0  0  0  0]
 [ 2  0 12  0  0  0  0  0  1  1]
 [ 1  0  1 14  2  0  0  0  0  0]
 [ 0  0  0  0 13  0  1  0  2  0]
 [ 0  0  0  0  0 18  0  0  0  0]
 [ 1  0  0  0  0  0 13  0  0  1]
 [ 0  0  0  1  0  1  0 11  3  0]
 [ 0  0  1  1  0  0  0  1 15  0]
 [ 3  0  0  1  0  0  0  0  0 12]]

SVC - Fold 5 Confusion Matrix:
[[14  0  0  0  0  1  0  0  1  0]
 [ 0  9  0  0  0  2  0  0  0  1]
 [ 0  0 13  0  0  0  0  2  0  1]
 [ 0  0  1 14  0  0  0  2  0  1]
 [ 1  0  0  1  7  0  2  2  3  0]
 [ 0  0  0  0  0 17  0  0  0  1]
```

```
 [ 0  0  0  0  0  0 15  0  0  0]
 [ 1  0  0  1  0  1  0 13  0  0]
 [ 1  0  1  0  0  1  0  1 13  1]
 [ 0  0  5  0  0  0  1  1  0 10]]

SVC - 5-FOLD CV RESULTS
Mean accuracy: 0.784
Std dev: 0.02
Scores: [0.784, 0.753, 0.802, 0.809, 0.772]
```

Applying automatic feature selection with k=50 decreased accuracy for both Logistic Regression and SVC. This indicates that the removed features still contained important predictive information.

8 CONCLUSIONS

This project presented the task of designing a complete and robust machine learning pipeline for music genre classification, starting from a deliberately noisy and imperfect dataset and improving a weak baseline model with an initial accuracy of approximately 37%. The final objective was not only to improve classification performance toward the 70–80% range. End-to-end pipeline overview

The implemented pipeline follows a structured and principled workflow: - Dataset exploration and diagnostics -Data cleaning and curation - Duplicate detection and removal - Noise filtering - Feature extraction and analysis - Feature selection and justification - Model selection, training, and evaluation

Each stage was explicitly validated through exploratory analysis or intermediate experiments to ensure that design choices were data-driven rather than arbitrary.

Data exploration and cleaning

Initial exploratory analysis showed that the dataset contained several quality issues, including:

- Unlabeled samples
- Empty or silent audio files
- Duplicate or near-duplicate recordings
- Noise-only or non-musical signals

Class distribution and audio-length histograms were used to assess dataset balance and consistency. While the original dataset was relatively balanced, successive cleaning steps introduced mild imbalance (notably in the classical genre), which was later addressed during model training.

Cleaning steps implemented

- Removal of unlabeled samples
- Removal of empty or silence-only audio
- Noise-only audio filtering, using conservative thresholds on:
- RMS energy (signal strength)
- Spectral flatness (noise vs harmonic structure)
- Duplicate detection and removal

To prevent information leakage and inflated performance, duplicate removal was performed in two stages:

1. Audio-level deduplication Compact audio signatures were constructed using MFCC summary statistics (mean and variance) and signal duration. These signatures were clustered using DBSCAN with a very small distance threshold to identify near-identical recordings.
2. Feature-level consistency check After feature extraction, exact duplicate feature vectors were identified. When duplicates shared the same genre label, only one instance was retained; conflicting-label duplicates were removed entirely.

This two-stage strategy improved data integrity and ensured that evaluation results reflected genuine generalization rather than memorization.

Feature engineering and representation

A diverse set of complementary audio features was extracted to capture different perceptual and structural aspects of music. All features were summarized using mean and variance statistics to produce fixed-length representations suitable for classical machine learning models.

Feature set overview:

1. MFCCs (20 coefficients) Capture the overall spectral envelope and timbral characteristics of audio.
2. RMS energy Encodes signal intensity and dynamic behavior over time.
3. Spectral contrast Measures differences between spectral peaks and valleys, capturing harmonic richness and timbre.
4. Zero Crossing Rate (ZCR) Reflects signal noisiness and percussiveness.
5. Spectral centroid Represents perceived brightness and frequency-weighted energy distribution.
6. Spectral rolloff Indicates how much energy lies in higher frequencies.
7. Chroma features Encode harmonic and pitch-class structure, invariant to octave shifts. This feature set intentionally combines time-domain, frequency-domain, and harmonic representations, ensuring that rhythm, timbre, dynamics, and tonality are all represented.

Feature analysis and intermediate validation

Several intermediate experiments were conducted to justify design decisions:

1. MFCC dimensionality selection MFCCs with 8, 13, and 20 coefficients were compared using Logistic Regression. MFCC(20) achieved the best accuracy (~71%), motivating its selection.
2. Correlation analysis Strong correlations between MFCC coefficients were observed, as expected due to their ordered spectral nature. Removing highly correlated features reduced accuracy, confirming that correlated MFCCs still carry useful discriminative information.
3. Class imbalance handling SMOTE oversampling was applied only to the training set, preserving test set integrity and avoiding data leakage.

These analyses ensured that feature and preprocessing choices were empirically validated rather than heuristic.

Final model selection

To identify the most effective classifier, multiple model families were systematically evaluated, including: - Logistic Regression, as a linear baseline - Support Vector Classifier (SVC), for its strength in high-dimensional spaces - Random Forest, as a non-linear, ensemble-based alternative

Each model was trained and evaluated using standardized feature scaling and cross-validation, allowing for a fair comparison of generalization performance.

In parallel, automatic feature selection was explored by evaluating different combinations of extracted features (e.g., MFCCs, spectral contrast, tempo-related features, RMS, and others). This process revealed that not all features contributed equally and that certain combinations consistently outperformed larger, more complex feature sets.

The final classifier chosen was a Support Vector Machine (SVC) trained on the combined feature set, incorporating MFCCs and all complementary features described above.

Why SVC?

- Well-suited for medium-sized datasets
- Effective in high-dimensional feature spaces
- Robust to correlated features
- Strong empirical performance in audio classification tasks
- Model training strategy
- Stratified train/test split
- SMOTE applied to the training set only
- Feature scaling via standardization
- Hyperparameter selection and evaluation using cross-validation
- Cross-validation ensured that the final reported performance reflected stable generalization rather than a favorable random split.

Final performance and outcomes

The final SVC model, trained on the best-performing bundle of features, achieved a substantial improvement over the baseline:

Baseline accuracy: ~37%

MFCC-only models: ~70–71%

Final combined-feature SVC model: significantly higher, approaching the target performance range

The improvement confirms that combining complementary audio features leads to better genre discrimination than relying on MFCCs alone.

[ ]:
```
!apt-get install -y texlive-xetex texlive-fonts-recommended␣
↪texlive-plain-generic
!jupyter nbconvert --to pdf /content/drive/MyDrive/Colab\ Notebooks/TEO_finale.
↪ipynb
```